sGLP: Simulating People with Stochastic Agents

Ehud Shapiro
London School of Economics and Weizmann Institute of Science
UK and Israel

ABSTRACT

Grassroots platforms are designed to be owned, operated and governed by their participants, and thus are constituted from person-to-person smartphone apps that are run by the participants. A grassroots app is specified using a grassroots social contract, which is compiled through an abstraction cascade into a volition-guarded Grassroots Logic Program (vGLP), a multiagent concurrent logic programming language. A vGLP program in turn specifies a smartphone UI via which people express their volitions and thus control the app. Prudence dictates large-scale evaluation and testing of grassroots platforms prior to deployment, but using the many people needed to do that is a contradiction in terms. Hence another solution is needed.

Here, we present stochastic GLP (sGLP), designed to allow grassroots platforms to be simulated at scale from the vGLP programs that deploy them, with stochastic agents in place of people. sGLP is GLP with a rate extension for specifying stochastic behaviour. A stochastic agent is an sGLP program replacing both the vGLP UI and the person behind it. It is typed by the dual of the interactive type of the UI and specifies a stochastic profile of a person's behaviour.

We demonstrate sGLP on the grassroots social graph, with a population of men and women drawn from four stochastic profiles — homophile or indifferent, wary or sociable — and on grassroots currencies, with four profiles — spenders or savers, cautious or generous — and report what the mix of profiles does to the social graph that forms and to the currencies that circulate.

We prove that every run of a vGLP program in which sGLP agents replace the UI and the people behind it is a vGLP run. We implemented sGLP in GLP by Gillespie's method, and ran reproducible simulations with it.

CCS CONCEPTS

• **Theory of computation** → Operational semantics; • **Software and its engineering** → Constraint and logic languages; Concurrent programming languages; • **Computing methodologies** → Discrete-event simulation; Agent / discrete models; • **Mathematics of computing** → Markov processes.

KEYWORDS

Stochastic Simulation, Gillespie Algorithm, Agent-Based Simulation, Operational Semantics, Concurrent Logic Programming, GLP, Volitional Programming, Grassroots Platforms

1 INTRODUCTION

Grassroots platforms [30, 32, 33] are designed to be owned, operated and governed by their participants, and thus are constituted from person-to-person smartphone apps that are run by the participants. A grassroots app is specified using a grassroots social contract [11], which is compiled through an abstraction cascade [11, 38] into a volition-guarded Grassroots Logic Program (vGLP) [38], a multiagent concurrent logic programming language [27, 34, 37]. A vGLP program in turn specifies a smartphone UI via which people express their volitions [15] and thus control the app. In vGLP a procedure may put a question to its person, and a clause that reads the answer reduces only after the person has given it, so the transitions of the platform are guarded by the volitions of its people [15]. Each question has an interactive type, and the UI is derived from these types, a widget per question [38]. The program is compiled onto GLP [34, 36] and deployed as a mini-app of the grassroots super-app [8], on iPhone and Android [12].

What such a platform does at scale — whether a grassroots social graph [31] coalesces from separate groups into one, whether grassroots currencies [29, 35] circulate — depends on what the people using it do, and cannot be found out by running the app with a handful of them. Prudence dictates large-scale evaluation and testing of grassroots platforms prior to deployment, but using the many people needed to do that is a contradiction in terms. Hence another solution is needed.

Here, we present stochastic GLP (sGLP), designed to allow grassroots platforms to be simulated at scale from the vGLP programs that deploy them, with stochastic agents in place of people. sGLP is GLP with a rate extension for specifying stochastic behaviour. A rated goal is reduced at a time drawn, when it is spawned, from the exponential distribution with its rate; the machine runs to rest between two such times, so simulated time advances only with them; and the law of a run is that of Gillespie's first reaction method [10], which we prove equivalent here to the direct method, since a spawned goal is never withdrawn (Section 2). A stochastic agent is an sGLP program replacing both the vGLP UI and the person behind it, typed by the dual of the interactive type specifying the UI, with a stochastic profile of a person's behaviour. A profile is such a program, named; a population draws its persons, each with a key, a role and attributes, and for each person a profile for each dimension of their role; and a person sees the program's output as their phone shows it (Section 3).

We demonstrate sGLP on the grassroots social graph [31], with a population of men and women drawn from four stochastic profiles — homophile or indifferent, wary or sociable — and on grassroots currencies [29, 35], with four profiles — spenders or savers, cautious or generous — and report what the mix of profiles does to the social graph that forms and to the currencies that circulate (Sections 5 and 6). The social graph is the platform on which the other grassroots platforms are built, and the currencies are coins among friends in it. Each demonstration runs 100 agents for a simulated year, over nine mixes of profiles and twenty seeds, and the social graph runs at 1,000 agents for a simulated year.

We prove that every run of a vGLP program in which sGLP agents replace the UI and the people behind it is a vGLP run. So what a

simulation reports is something the deployed platform can do (Section 3). We implemented sGLP in GLP by Gillespie's method [10], and ran reproducible simulations with it. The implementation is as the stochastic pi-calculus [23] was implemented in Flat Concurrent Prolog [24, 25, 39]: a transformation of sGLP programs into GLP, whose engine holds the simulated clock and draws the next event by Gillespie's method, stepped by a monitor written in GLP over a guard that succeeds when the machine rests; every draw is from the run's seed, so a run is reproducible (Section 4). Section 7 reviews related work and Section 8 concludes.

2 SGLP: STOCHASTIC GLP

A stochastic agent is a program that replaces the UI of a vGLP program and the person behind it: it holds the person's end of each question and answers it, as the widget does on the phone. sGLP is the language such a program is written in: GLP [34, 37] with a rate on a goal, which runs in simulated time. This section defines the language; Section 3 puts a person written in it in the place of the person of a vGLP program.

2.1 Syntax

An sGLP program is a typed GLP program [37] with a rate on a goal. Its draws are GLP's: `random(`$S?$`, `$N?$`, `K`, `S_1`)` assigns K an integer uniform from 1 to N, drawn from the seed S, and S_1 the next seed, and `split_random_seed(`$S?$`, `S_1`, `S_2`)` assigns S_1 and S_2 two seeds made from the seed S, the draws from S, S_1 and S_2 being independent streams [34].

```
<rated_goal> ::= <goal> @ <rate>
<rate>       ::= <positive_real> / <time_unit>
<time_unit>  ::= second | minute | hour | day | week | year
```

Definition 2.1 (Rated Goal). A *rated goal* A `@` r in the body of a clause is a goal A with a *rate* r, a positive real per unit of time.

A rated goal is spawned as any body goal is, and is reduced only once its time has come, which is drawn when it is spawned from the exponential distribution with its rate.

A choice among alternatives with given probabilities is a draw and a guard on it in each clause, as the profiles of Sections 5 and 6 show.

2.2 Operational Semantics

Throughout, N is an sGLP program, $P \subset \Pi$ a finite set of agents, and c a configuration of the maGLP transition system over P and N [34] (Definition E.8), whose transitions are Reduce, Communicate and Cold-call. sGLP adds to a configuration a clock and the activation times of its rated goals, and to the transitions one that advances the clock.

Definition 2.2 (Configuration, Pending, Quiescent). A *configuration* of sGLP over P and N is a triple (c, t, α): c a configuration of the maGLP transition system over P and N, $t \geq 0$ the *simulated time*, and α a partial function from the goals of c to *activation times* $\alpha(A) \geq t$, the goals in its domain being *pending*. It is *quiescent* if no Communicate, no Cold-call and no Reduce of a goal that is not pending is enabled in c.

Definition 2.3 (sGLP Transition System). The *sGLP transition system* over P, N and a configuration c^0 of the maGLP transition system over P and N has the configurations of Definition 2.2, the initial configuration $(c^0, 0, \emptyset)$, and the transitions $(c, t, \alpha) \rightarrow (c', t', \alpha')$:

(1) **Machine:** $c \rightarrow c'$ is a Communicate or Cold-call transition, or a Reduce of a goal A that is not pending; $t' = t$; and α' is α with $\alpha'(B) = t + \tau$ for each rated goal B `@` r in the body the Reduce spawns, τ drawn from the exponential distribution with rate r, the draws independent.
(2) **Release:** (c, t, α) is quiescent, A is a pending goal with $\alpha(A)$ least, $c' = c$, $t' = \alpha(A)$, and α' is α without A.

Definition 2.4 (sGLP Run). An *sGLP run* of N over P from c^0 is a sequence of configurations of the sGLP transition system over P, N and c^0, the first initial and each a transition of the one before, such that its configurations of the maGLP transition system, with each repeated one taken once, form a proper safe computation of that system from c^0 (Definitions E.1 and E.17, the latter read for a computation, each configuration (G, σ) read as $(\emptyset, G, \sigma)$); an *sGLP run* of N over P is one from the initial configuration of the maGLP transition system over P and N.

Simulated time advances only at a Release: a machine step takes no time, and the clock jumps to the earliest activation time once the machine has nothing to do at the present time. A released goal is an ordinary goal: where its input has not arrived it suspends and reduces when it does, with no further delay.

Definition 2.5 (History, Non-Anticipating). The *history* of an sGLP run up to a configuration is the sequence of its configurations up to it, each with its activation times replaced by the set of its pending goals. A run is *non-anticipating* if each Machine transition it takes is chosen as a function of its history up to that transition.

Proposition 2.6 (Time to the Next Release). *Let (c, t, α) be a quiescent configuration of a non-anticipating sGLP run with pending goals $A_1, \ldots, A_m$ of rates $r_1, \ldots, r_m$, and $R = \sum_i r_i$. Conditional on its history up to it, the next Release comes after a time exponentially distributed with rate R, and releases A_i with probability r_i/R, independently of that time.*

Proof. $\alpha(A_i) = t_i + \tau_i$, with $t_i \leq t$ the time A_i was spawned and τ_i exponential with rate r_i, the τ_i independent of one another and of every other draw of the run. The history up to (c, t, α) depends on τ_i only through $\alpha(A_i) \geq t$: the run is non-anticipating, and each Release before it released a goal of least activation time. So conditional on it $\alpha(A_i) - t$ is exponential with rate r_i, by memorylessness, and the $\alpha(A_i) - t$ are independent. The minimum of independent exponential variates with rates $r_1, \ldots, r_m$ is exponential with rate R, and is the i-th with probability r_i/R, independently of its value. □

This is Gillespie's first reaction method [10], and the proposition is its equivalence to the direct method, which holds here because a spawned goal is never withdrawn.

3 SIMULATING A VGLP PROGRAM

On the phone, the person's end of each interactive variable is held by a construct process, which has the question's widget drawn and assigns the person's answer; the widget's type is the dual of the interactive type, writing where the program reads and reading where the program writes [38]; and the program's output to its

person is drawn as construct `0` (Appendix E). In a simulation the person's end is held by a relay, which hands a copy of the question to a goal of a person procedure of the dual type and passes the goal's answer on to the program; the program's output is held by a screen, from which the person's goals read what the person has been shown; and the population is drawn by a census, which gives each person a key, a role and attributes, and by a mix of profiles of person for each role, each profile a program. The vGLP program, its types and its clauses are those the phone runs.

```
<person_declaration> ::= <type> =::= <procedure_name> .
<profile>     ::= person <name> .
                  <person_declarations> <program>
<run>         ::= run <persons> [ <mix_list> ]
                  until <time> seed <integer> .
<persons>     ::= <integer> agents | <procedure_name>
<mix_list>    ::= <mix> | <mix> , <mix_list>
<mix>         ::= <dimension> <of_role> ~
                  ( <name> : <probability> ; ... )
<of_role>     ::= <empty> | of <role>
```

The definitions and results of vGLP [38] that this section and Theorem 3.15 rest on — interactive types, the vmaGLP transition system, the canonical compilation and its mapping back to the source — are recalled in Appendix E. Throughout, M is a vGLP program, $\mathcal{T}(M)$ the set of its interactive types, each in writer or reader mode, and $\lceil M \rceil$ its canonical compilation (Appendix E), which gives the interactive types distinct functors, A being the type of its asks.

Definition 3.1 (Person, Census). A *person* is a ground term of the form `person`(K, ρ, V), of a *key* K, a *role* ρ, a constant, and *attributes* V, `Person` being a type of such terms; and a *census* is a procedure w of a GLP program, declared `procedure` w`(Integer?, List(Person))`, such that, for each integer s, every correct run of the maGLP transition system over one agent and that program, with the initial local state $(\{w(s, L)\}, \emptyset)$, assigns L the same finite list of persons with distinct keys, the persons w *draws* from s.

A census draws the persons of a run: how many there are, their keys, their roles — a parent or a child, say — and their attributes — the village and the family of a person, and the persons they know at the start. The run declaration `run` n `agents` has the census that draws, from every seed, the persons `person(`K_a`, agent, [])` for $a = 1, \ldots, n$, K_a the key of agent a (Appendix B), and a mix with no role is of the role `agent`.

Definition 3.2 (Dual, Person Procedure, Person Declaration). The *dual* of an interactive type T is $T^\circ := T'?$ where $T = T'$ and $T^\circ := T'$ where $T = T'?$, with each `Stream(`X`)` in T' replaced by `Endless(`X`)`, the type `Endless(X) ::= [X  Endless(X)]`|, so that a person procedure ends no stream, as a person on the phone ends none; a *person procedure* of T in an sGLP program N is a procedure p of N declared `procedure` p`(`T°`, Person?, Integer?)`; and a *person declaration* is a pair (T, p), written T `=::=` p, of an interactive type $T \in \mathcal{T}(M)$ and a person procedure p of T.

The person's end of an interactive variable is of the dual type, and a person procedure takes a copy of it, of the same type, the person and a seed. A person procedure, and a procedure it calls, may call the goal `seen(`L`)`, which assigns L the elements of the program's output that the person has seen, the last first (Definition 3.8).

Definition 3.3 (Profile, Dimension). A *profile* over M is a triple $\phi = (name, N, \Delta)$ of a name, an sGLP program N and a finite set Δ of person declarations with distinct interactive types, each naming a person procedure of N, $\mathcal{T}(\phi)$ being the set of those types; a *dimension* over M is a triple (d, ρ, Φ) of a name, a role and a finite set Φ of profiles over M with one set of interactive types, $\mathcal{T}(\Phi)$; and dimensions over M are *disjoint and covering* if, for each role ρ of one of them, the sets $\mathcal{T}(\Phi)$ of those of role ρ are pairwise disjoint with union $\mathcal{T}(M)$.

Definition 3.4 (Stochastic Person, Population). For disjoint and covering dimensions $D = ((d_1, \rho_1, \Phi_1), \ldots, (d_k, \rho_k, \Phi_k))$ over M and a role ρ, with $j_1 < \cdots < j_m$ the j for which $\rho_j = \rho$, a *stochastic person* of role ρ over D is a tuple $(\phi_1, \ldots, \phi_m)$ with $\phi_l \in \Phi_{j_l}$ for $l = 1, \ldots, m$; and a *population* over M is a tuple (D, μ, w, s) of such dimensions D, a tuple $\mu = (\mu_1, \ldots, \mu_k)$ of probability distributions, μ_j on Φ_j, a census w and an integer s, its *seed*; and, for an integer s_w, its *persons* from s_w are the persons $h_1, \ldots, h_n$ that w draws from s_w, and its *agents* are $P = \{1, \ldots, n\}$, the person of agent a being h_a.

A stochastic person therefore declares each interactive type of M exactly once, by one of its profiles. So a parent can have two dimensions and a child one.

Definition 3.5 (Well-Formed). A population is *well formed* if its census draws at least one person from every seed, each of the role of a dimension of D, each person procedure its profiles declare is declared with the type its declaration requires, the predicates of the profiles' programs and of the census are distinct from those of $\lceil M \rceil$, a predicate or a type that two of these programs define has one definition in both, $\lceil M \rceil$ defines no procedure or type named `screen`, `seen`, `Seen` or `Person`, the profiles' programs and the census define none named `screen`, `seen` or `Seen`, and μ_j sums to one over Φ_j for each j.

The transformation of Section 4 refuses a population whose dimensions, profiles and distributions are not well formed, and the run refuses a census that draws no person, or a person of a role of no dimension; that a census draws the same persons in every run is its author's to ensure. A run declaration draws from the population's seed its persons, by its census, and for each agent a stochastic person of the role of its person — one profile per dimension of that role, by that dimension's distribution — and a seed; Section 4 says how. For the rest of the section, (D, μ, w, s) is a well-formed population over M with persons $h_1, \ldots, h_n$ and agents P from an integer s_w, which the run declaration draws from s, and, for each $a \in P$, $\pi(a)$ is a stochastic person over D of the role of h_a, s_a an integer and g_a a goal of M, its initial goal, with g'_a the goal g_a extended with an ask stream U_a, as in $\lceil M \rceil$. Where M defines `Output` (Appendix E), O_a is a writer of g_a of type `Stream(Output)`, given with g_a as the program's output to its person, its reader occurring nowhere in g_a; where M does not, $O_a?$ stands for `[]` and `Output` for a type of no term.

A Communicate removes the assignment it delivers from the readers substitution of its agent (Definition E.7). For the rest of the section, the term *assigned* to a reader is the term of the assignment a transition of the run added to it, whether or not a Communicate has since delivered it; the readers substitution σ_a of an agent a in a

configuration holds the assignments not yet delivered. A variable is *reachable* in a configuration if it occurs in a goal of the configuration, or in the term that a readers substitution of the configuration assigns to a reader reachable in it.

Definition 3.6 (Copy, Received). For $T \in \mathcal{T}(M)$, t the functor $\lceil M \rceil$ gives T (the canonical compilation, Appendix E), and $relay_T$ a procedure declared `procedure` $relay_T(T^\circ,\ T)$, let $relay_T(R, Y)$ be a goal with R the person's end X of the interactive variable of an ask `ask(`T`,` $t(X)$`)`, a writer whose reader the person's end X is assigned, or a reader assigned one of these, and Y one end of a fresh variable pair, the *copy*, whose other end is of type T°. A variable is *received* through R if it is R, a variable of a term assigned to a variable so received, or the pair of one, and is *of the copy* if it is Y, a variable of a term assigned to a variable of the copy, or the pair of one.

Definition 3.7 (Relay). The goal $relay_T(R, Y)$ is a *relay* if each reduction of it and of the goals descended from it (*i*) assigns a variable received through R only where it is a writer at a position the person writes, and then either the reader of a fresh writer, which *takes the place* of the writer assigned, or its *answer*, at most one answer, and only where the reader of the writer assigned is reachable in the configuration: where the writer's place in the copy holds a term ground but for variables at its stream and list positions and at the positions the program writes, a list cell where the writer is of a stream type and a list cell or `[]` where it is of a list type, that term with a fresh reader at each stream and list position the person writes, but where the copy ends a list there with `[]`, and a fresh writer at each outermost position the program writes; (*ii*) assigns a variable of the copy at a position the program writes either the reader of a fresh writer, which *takes the place* of the variable assigned, or, only at the place of a reader received through R whose term, each reader in it replaced, repeatedly, by the term assigned to it, has no reader, once Communicates have delivered the assignments of that reader and of each reader so replaced to the relay or goals descended from it, at one such place in a reduction and at none in a reduction that answers, and then with that term, each writer in it replaced by a fresh writer, that reader and each reader so replaced being then *presented*; (*iii*) puts no variable received through R in a term it assigns to a variable of the copy; and (*iv*) passes on to the goals it spawns each variable received through R that its goal holds and does not assign, but for a writer at a position the person writes whose reader is not reachable in the configuration, which it may *discard*, passing it on to no goal, and the writer of each fresh reader and the reader of each fresh writer it assigns a received variable with.

A relay holds the person's end of a question, shows the copy the program's output once it is written whole, as a construct process draws it, and passes the copy's answer on to the program once it is complete, as a construct process forms an answer from the person's input; an answer of a stream or a list type is passed on one element at a time. A writer passes from goal to goal of a relay as GLP passes any writer on, by the reader of a fresh writer that takes its place. A relay answers a question only while the question's reader is reachable, and may discard a question whose reader is not, the program having withdrawn it, as a construct process removes a widowed question [38]; a discarded question is never answered.

Definition 3.8 (Screen, Seen). The *screen* is the procedure `screen` with the declaration and the clauses

```
Seen ::= seen(List(Output)?).
procedure screen(Stream(Output)?, List(Output)?,
                 Stream(Seen)?).
screen([E|O], Os, Rs) :-
    ground(E?) | screen(O?, [E?|Os?], Rs?).
screen(O, Os, [seen(Os?)|Rs]) :-
    ground(Os?) | screen(O?, Os?, Rs?).
screen([], _, []).
```

a reduction of a goal of it by the first clause *showing* the element `E`. A procedure of an sGLP program *reaches the screen* if it is `seen`, declared `procedure seen(List(Output))`, which a profile's program may call and does not define, or a person procedure, or a clause of it calls a procedure that reaches the screen; and the *screened* form of an sGLP program has each procedure that reaches the screen with one more argument, last, of type `Stream(Seen)`, its *screen stream*, which its clauses give the calls in their bodies as the clauses of $\lceil M \rceil$ give ask streams (Appendix E); the screened `seen` has the clause `seen(L?, [seen(L)])`.

The screen holds the program's output to its person and shows its elements in order, each once it is ground, as construct `0` draws them. A goal `seen(`L`)` of a person's program sends the screen a request on its screen stream, and the screen assigns L at once the elements it has shown, the last first.

Definition 3.9 (Person Process). The *person process* of a stochastic person ϕ over D is the procedure $person_\phi$, written `person_phi`, with the declaration and the first clause

```
procedure person_phi(Stream(A)?, Person?,
                     Stream(Seen), Integer?).
person_phi([], _, [], _).
```

and, for each person declaration (T, p) of a profile of ϕ, t the functor $\lceil M \rceil$ gives T, the clause

```
person_phi([ask(T, t(E))|As], H, R?, S) :-
    ground(H?) |
    split_random_seed(S?, K, S1),
    p(Y?, H?, K?, R1), relay_T(E?, Y),
    merge(R1?, R2?, R), person_phi(As?, H?, R2, S1?).
```

with p the procedure of the screened form of the profile's program, `E` and `E?`, and `Y` and `Y?`, exchanged where T is in reader mode, where the clause assigns the writer the ask carries the reader `E?`, so that `E` *takes its place*, and $relay_T$ being a procedure whose goals $relay_T(R, Y)$ are relays (Definition 3.7), its predicate and those of the procedures it reaches distinct from those of $\lceil M \rceil$ and of the profiles' programs.

The end the ask carries is the writer where T is in reader mode and the reader where it is in writer mode (the asking clause, Appendix E). On each ask the person process hands the person's end of the question to the relay it spawns, and the other end of the copy to the person procedure its profile declares for the question's type, with the person, a seed split from its own and a screen stream, which it merges into the screen's.

Definition 3.10 (Simulation Program). The *simulation program* of (M, P, π) is the sGLP program N whose clauses are those of $\lceil M \rceil$

other than the construct processes' and the dispatcher's; those of the screened forms of the programs of the profiles of $\pi(a)$ for every $a \in P$; those of $person_{\pi(a)}$ for every $a \in P$, with the procedures $relay_T$ they call and the procedures these reach; and those of `screen`, the screened `seen` and `merge`; and whose *initial configuration* c^0 has at each $a \in P$ the resolvent $(\{g'_a,\ person_{\pi(a)}(U_a?, h_a, R_a, s_a),\ screen(O_a?, [], R_a?)\},\ \emptyset)$, R_a a writer occurring nowhere else.

A *stochastic agent* is an agent of the sGLP transition system over P, N and c^0: it runs the platform's program as the phone does, with the person process, the relays, the screen and the programs of its stochastic person in place of the dispatcher, the construct processes and the person behind them. A run declaration `run ... [...] until` T `seed` s runs the simulation program of M over the agents of the persons its census draws, with the stochastic persons and seeds its population draws from s, until the simulated clock reaches T; the initial goal of each agent is stated with the program, as Sections 5 and 6 state theirs. A person's first acts — befriending the persons they know at the start, say — are their answers to the program's first questions, given as every answer is. Every seed of a run is split from the population's seed, so the draws of a run are reproducible from that seed, and the run itself is reproducible where the machine's schedule is deterministic. For the rest of the section, $r = (c^1, t_1, \alpha_1), (c^2, t_2, \alpha_2), \ldots$ is an sGLP run of N over P from c^0.

Definition 3.11 (Platform Goal, Person Goal, Screen of an Agent). A goal in a resolvent of r is a *person goal* if it is the goal $person_{\pi(a)}(U_a?, h_a, R_a, s_a)$ or the goal $screen(O_a?, [], R_a?)$ of c^0, for some $a \in P$, or a goal a Reduce of a person goal spawns, and a *platform goal* otherwise; and the *screen* of a is that goal of `screen` and each goal of `screen` a Reduce of the screen of a spawns.

Definition 3.12 (Interactive Variable, Interface Variable, Ask-Stream Variable, Question of a Writer). The *interactive variables* of an agent a in r are the variables X of the asks `ask(`T`, `$t(X)$`)` written by the reductions of asking clauses at a in r; the *interface variables* I_a of a in r form the least set closed under pairs that contains the interactive variables of a and the variables of every term assigned, by a transition $c^i \to c^{i+1}$ of r, to a writer $X \in I_a$; the *ask-stream variables* of a are the variables at the ask-stream argument of a platform goal of a and at the arguments of the `merge` goals the compilation adds, in a configuration of r, the variables a `merge` clause binds to the elements it moves, and the tail variable of each list assigned to one of them; the *question* of a writer of I_a is the question of the writer whose place it takes (Definitions 3.7 and 3.9), where it takes one, and the writer itself otherwise.

Definition 3.13 (Log). For $i \geq 1$ and $a \in P$, let θ_i be the restriction to I_a of the assignments the transition $c^i \to c^{i+1}$ adds at a but for those by which a writer takes another's place, each read as an assignment to the question of its writer, the identity substitution where it adds none; the transition is *logged* if it is a Reduce of a person goal of a and θ_i is not the identity, its *entries* being the triples $(t_i, a, s\{X := v\})$ for the assignments $X? := v$ of θ_i, s being the term in which X was handed to the person — the ask's term where X is an interactive variable, the term of Definition 3.7 (*ii*), its writers not replaced, with which a relay of a assigned the copy, where X is among its variables, and X itself where a person goal introduced it — and every variable of $s\{X := v\}$ replaced by the anonymous variable, or if it is a Reduce of the screen of a showing an element e, its *entry* being $(t_i, a, \texttt{output(}e\texttt{)})$; and the *log* of r is the sequence of the entries of its logged transitions, in the order of r.

An entry of the log is an answer with the question it answers, as the person was shown it, the positions still open anonymous, or an element of the program's output, as the person was shown it, so the log is what the people were asked, what they were shown and what they answered, one entry per answer and per element shown, an answer of a stream type one element at a time; a run adds nothing to the program it runs. Reading the logs and analysing the runs — counting, aggregating, comparing populations, plotting — is separate software, outside the language and outside this paper.

A simulated run is a run of the platform. The mapping that says so reads the platform's configuration off the platform goals, as vGLP's compilation mapping (Definition E.21) reads it off the compiled goals, with the person goals in the place of the construct processes and the dispatcher.

Definition 3.14 (Simulation Mapping). The *simulation mapping* σ of r takes a configuration (c, t, α) of r, with σ_a the readers substitution of a in c, to the configuration of the vmaGLP transition system over P and M with, at each agent $a \in P$, the agent state (H, G, σ_r):

- G = the multiset of the platform goals of a in c, each read back to its source with its ask stream dropped: $q(S_1, \ldots, S_n, D)$ as the n-ary goal $q(S_1, \ldots, S_n)$, $q_1(S_1, \ldots, S_n, X?, D)$, and $q_1(S_1, \ldots, S_n, X, D)$ as the asked goal, with the identifier of the goal its asking clause reduced, and any other platform goal as itself; the `merge` goals the compilation adds and the person goals contributing nothing; and in every goal an asking clause's pair for an argument read as that argument, as in Definition E.21, and a writer of I_b, for any $b \in P$, and its reader read as its question and its question's reader (Definition 3.12);
- H = the variables of I_a that occur in a person goal of a other than its screen or in a `merge` goal the compilation adds of a, in c, in a term of an assignment of σ_a to an ask-stream variable that no Communicate of r up to (c, t, α) has delivered, or in a term of an assignment other than an answer that a Reduce of a person goal added to σ_a for a reader a person goal of a other than its screen holds, and the writers of I_a that a relay of a has discarded up to (c, t, α) (Definition 3.7), but, for each reader $Z?$ of I_a at a position the program writes that is received through the R of a relay of a by the assignments of c (Definition 3.6), is not presented up to (c, t, α) (Definition 3.7) and is in the *contents* of no other such reader — the term assigned to it in c, the contents of the readers in that term, and the writers that take the place of a writer in them, repeatedly — $Z?$ itself and none of the variables of its contents; read as in G, each with its type, a writer where the person writes and a reader where the program writes; and, for each reader of I_a at whose place a relay of a has assigned the copy up to (c, t, α), its assignment with the term of Definition 3.7 (*ii*), its writers not replaced, shown;
- σ_r = the assignments added by a Reduce of a platform goal of a to variables other than ask-stream variables and the

variables of the pairs by which an asking clause passes its arguments on, an asking clause's assignment $W? := S_l?$ of an output position read through as in Definition E.23, and the answers added by a Reduce of a person goal of a to writers of I_a, read as in G: each while it is in σ_a, and, where a Communicate delivered it to a person goal of an agent, until the reader it assigns is presented (Definition 3.7), a reader a screen holds being never presented.

$\sigma(r)$ is as in Definition E.2.

A screen contributes nothing to σ, so its showing an element is a stutter; the element is in σ_r, added by the platform goal that wrote it.

Let r be an sGLP run of the simulation program N of (M, P, π) over P from its initial configuration c^0, π over a well-formed population, and σ the simulation mapping of r.

Theorem 3.15 (Soundness). *$\sigma(r)$ is a proper safe computation of the vmaGLP transition system over P and M from $\sigma(c^0)$ in which every Answer is the image of a Reduce of a person goal.*

Corollary 3.16. *Where every g_a is the initial goal* `agent(ch(_?,_),ch(_?,_))` *of M, with O_a one of its writers where M defines* `Output`*, $\sigma(r)$ is a proper safe run of the vmaGLP transition system over P and M.*

The proofs are in Appendix A. What a simulation reports is therefore something the deployed platform can do, with the people's answers given by the stochastic agents: a person procedure's goal computes the answer into its copy of the question, and the relay assigns the program's variable with it. Simulated time advances only at a Release, and $\lceil M \rceil$ has no rated goal, so it advances only with the persons' acts. The converse — that every run of the platform is the image of a simulated run under some population — is not claimed: a simulation shows what its profiles do.

4 IMPLEMENTATION

sGLP is implemented in GLP by Gillespie's method [10], as the stochastic pi-calculus was implemented in Flat Concurrent Prolog [24, 25, 39]: by a transformation of sGLP programs into GLP, a monitor process written in GLP, and two things from the machine, a simulated clock with its queue of pending goals, and a guard that succeeds when the machine has come to rest. The compiler and the type checker of GLP [36] are not changed; the engine has the clock and the queue [34], as the emulator of Flat Concurrent Prolog had them for the stochastic pi-calculus [24].

The guard. A machine step carries no simulated time, so the clock may move only when no transition of the machine is possible; with no way to learn that, the monitor cannot know when to move it. GLP has the guard `when_idle`, which suspends while the machine has a Reduce or a Communicate to make and succeeds when it has none [34]. It is sound for one machine and not for many — in a multiagent system a communication changes an agent's configuration without that agent taking a transition — which is why the simulator hosts the whole population in one machine. SpiFcp's monitor did the same, waiting on the Logix system's `idle_wait` before its next draw [39].

The clock. GLP's engine has a simulated clock, in seconds, and a queue of pending tokens, each with an activation time, and three body goals over them [34]: `start_clock(Seed)` sets the clock to zero and seeds the clock's generator; `rated(Rate, Tok)` makes the token `Tok` pending at the clock plus a delay drawn from the exponential distribution with rate `Rate` a second; and `step_clock(Horizon, Step)`, where a token is pending whose activation time T is the least and at most the horizon, sets the clock to T, binds that token to `go(`T`)` and `Step` to `released(`T`)`, two tokens of one time released in the order they were made pending, and otherwise binds every pending token to `stop` and `Step` to `stopped(`C`)`, C the clock. This is Gillespie's algorithm [10] in its first reaction form (Section 2): the time is drawn when the goal is spawned, and the earliest is released.

The monitor. The monitor is a GLP goal holding the clock of the last step and the run's horizon. Its step clause is guarded by `when_idle` and calls `step_clock`: when the machine rests, every rated goal spawned has made its token pending, so the queue is that of the configuration, and the step releases the least activation time, which is Release. A `released` step is followed by the next; a `stopped` step, the least activation time being past the horizon or no token pending at rest, ends the monitor, which writes the clock. Its input is one stream, a mutual reference of GLP's into which the person processes and the screens append the request `time(T)`, which it answers at once from the clock. The module is Appendix C.

The transformation. A profile is translated into GLP: its person procedures take one more argument, the screen's reference, which the procedures they call that reach the screen (Definition 3.8) take as well; a goal `seen(L)` becomes the request `seen(L)` appended to that reference; and a rated goal `G @ r` becomes the goal `rated(R, Tok)`, `R` the rate `r` as a number a second, followed by `G` with `Tok?` as an added first argument, matched by a clause that takes the goal only once the engine has bound the token to `go(T)` and by one that ends it once the engine has bound it to `stop`, a rated goal's procedure taking inputs only, which the transformation checks; a rated goal that writes an answer receives its writer inside a term it reads. A person declaration `T =::= p` becomes a clause of the person process, and the run declaration becomes the initial goals of the run, with the census it names. The transformation is a GLP program: it reads an sGLP program in a term form, one GLP term per declaration and clause, checks that the dimensions, profiles and distributions of its population are well formed (Section 3), and prints GLP source, as the compiler of grassroots social contracts reads contracts [11]; the syntax printed in this paper is the surface form of that term form. The profiles and populations of Sections 5 and 6 are translated by it; the vGLP program's compilation and the harness of a run are written by hand. GLP's own `Goal@Agent`, which places a goal at an agent in a boot clause, is untouched.

The person process. In vGLP's canonical compilation an asking clause writes an ask — the question's interactive type and the person's end of its variable — on an ask stream that the compiler threads through every procedure that reaches a question, and the dispatcher, reading the initial goal's ask stream, spawns on each ask the construct process that has the bridge draw the widget [38]. In a simulation the dispatcher's place is taken by a GLP process, the *person process* of the agent: it holds the agent's person, its stochastic person, one profile per dimension of its role, and a seed, and on each ask it spawns the goal $p(Y, H, S, Scr)$ of the person procedure p that

its profile declares for the type, with Y the person goal's end of a copy of the question, H the agent's person, S a seed it splits from its own, and *Scr* the reference of the agent's screen, and the relay of the question (Definition 3.7), its answer and present procedures, by the question's moded type. The person goal answers into its copy, and the relay hands the program's variable the answer once the positions the person writes are ground but for those of a stream or a list, which it hands over one element at a time, with fresh writers at the outermost positions the program writes, whose readers it keeps, showing the person goal the program's terms there the same way — vGLP's Answer, which waits on nothing the program writes — and that is how the person process comes to log it. The relay hands over an answer only while the reader of the program's variable is reachable: the first clause of each answer procedure is guarded by GLP's `widowed_writer`, which succeeds once the reader paired to a writer occurs in no goal [34], and ends the procedure, so a question the program has withdrawn is discarded, with no answer handed over and none logged. The other way, the relay shows the person goal the question once the positions the program writes are ground, the positions the person writes inside them left open, each answered as a question of its own — vGLP's Communicate of the program's output to the person. Every term crossing between the program and the person goal, in either direction and at every depth, crosses the relay so, a question the program puts inside an answer included. The platform's program compiles as for the phone.

The screen. The agent's screen is a GLP goal holding the reader of the program's output to its person, which vGLP's compilation draws as construct `0`: it shows its elements in order, each once it is ground, keeps the elements it has shown, the last first, and answers each request `seen(L)` at once with them (Definition 3.8). The requests of the agent's person goals reach it by the screen's reference, a mutual reference, as the time requests reach the monitor. A program that writes nothing to its person has a screen that shows nothing.

The run. The population runs as one program in one machine, and an agent is an attribution: its initial goal as the program states it, with the agent's number and the lists it is created with, its person process, its screen, and what they spawn. The agents are numbered $1, \ldots, n$, in the order of the persons the census draws. The harness's goals are the network between the agents, a mutual-write merge of their output streams and a tree of routing goals delivering each message to the input stream of the agent it names, and the monitor. Every seed of the run is split from the run's seed by `split_random_seed` (Section 2): one for the clock, which the monitor gives `start_clock`; one for the census, where the run names one; one for each agent, from which it draws its profiles, one per dimension of its role, and splits its person process's seed; and, in a person process, one for each person goal. So the draws of the clock, the census, the agents and the person goals are independent streams, and the run is reproducible where the machine's schedule is deterministic, as GLP's is.

The log. The person process and the screen write the log of their agent: one line for each answer a relay hands the program, with the simulated time, which they ask of the monitor, the agent, the interactive type and the term handed over, the positions the program writes as variables; and one line for each element of the output the screen shows, with the simulated time, the agent and the element. Every answer crosses a relay and every element shown crosses the screen. The harness names the log's sink when the run starts.

The implementation decisions beyond Sections 2 and 3 are in Appendix B. We claim, informally, that a run of the transformed program, up to the monitor's stop, is an sGLP run of the simulation program: the transformed clauses reduce as the source clauses do, a rated goal's token is bound only by `step_clock`, which the monitor calls only when `when_idle` has succeeded, which is quiescence, and which releases the least activation time, which is Release; the machine's schedule reads no activation time, so the run is non-anticipating; the screen's reference merges the requests of the person goals as the screen streams do; and the monitor's reductions, the harness's, and those of the person process and of the screen that ask the clock and write the log are stutters.

The claim, tested. Two tests in the repository check the claim where it bites. With a rated goal and an unrated goal that runs on, the unrated goal runs to rest before the rated one is released, whichever is spawned first: `when_idle` succeeds at quiescence and not before. With four pending tokens of rates 1/second, 2/second, 30/minute and 3600/hour and nothing else, the first 10,000 times between releases have mean 0.2175 s and variance 0.04677 s^2, against $1/4.5 = 0.2222$ and $1/4.5^2 = 0.04938$ of Proposition 2.6, 2.1% and 5.3% under.

The cost. Measured on one machine, under the load of other work, the social graph of Section 5 at its mix: 100 agents for a year is 15,055 answers in 26 s of wall-clock time, 1.7 ms an answer, at 85 MiB; 30 days at 100, 200 and 500 agents take 1.9, 2.1 and 3.2 s, 19, 11 and 6 ms an agent and 1.5, 0.9 and 0.5 ms an answer, at 85 to 89 MiB. On a second machine, about half as fast and with no other work, 1,000 agents for a year is 146,881 answers in 10,004 s, 68 ms an answer, at 888 MiB at the peak, the memory growing through the year. The draw of a population of 100 costs about 4 s besides.

How it was built. The implementation — the monitor, the person process, the two translations, the harnesses, the replays and the tests — was written by Claude (Anthropic) from this paper, by the Human–Science–AI methodology [11, 36]: the paper is the source of authority for coding, and every gap coding found was repaired in the paper before it was repaired in the code; the runs' analyses and the figures were made the same way.

5 EXAMPLE: THE GRASSROOTS SOCIAL GRAPH

The platform's agent offers friendship to a peer its person picks, and asks its person to approve each offer that arrives. The agents are numbered 1 to n; the first half are male and the rest female. Each agent is created knowing two other agents of its sex and two of the other sex, drawn from the run's seed, and starts by asking its person the first menu, by its exported procedure `start`; it comes to know other people through its friends.

```
Peer     ::= Constant.
YesNo    ::= yes ; no.
Peers    ::= [Peer | List(Peer)].
Choice   ::= same(Peer) ; other(Peer).
Menu     ::= menu(Peers, Peers, Choice?).
Approval ::= approval(Peer, YesNo?).
```

```
%% Show the person both lists, and hand them the choice.
procedure (Menu)*ask(Peers?, Peers?, Choice).
(menu(Ss?, Os?, C))*ask(Ss, Os, C?).

%% Show the person who offers, and hand them yes or no.
procedure (Approval)*respond(Peer?, YesNo).
(approval(From?, A))*respond(From, A?).
```

The agent's own question is `Menu`, in writer mode: `ask` writes the two lists at once, the people the agent knows when it asks, of its sex and of the other, and hands the person the choice, which stays open while offers come and go, and each offer the person makes is followed by a new menu. An arriving offer is `Approval`, also in writer mode: `respond` shows the person who offers and hands them yes or no, and a yes is sent back when they give it. The agent's clauses, in the repository[1], send an offer to the peer the person picks and ask again, the offer saying whether the two are of one sex, which the agent knows by the list it was picked from; ask the person to approve each offer that arrives, and send the offerer the person's yes, a no sending nothing; and on a yes the two agents send each other their friends, by sex, and each adds to the people it knows the other and the other's friends, but for itself and those it knows already. A person thus meets people through their friends, and the next menu shows them; the friends a friend makes later are not sent. Both interactive types are in writer mode, so their person procedures take readers.

The profiles are two dimensions of two. `one_of(Xs?, S?, X)` draws `X` uniformly from the list `Xs` by `random` from the seed `S`, the list being non-empty by its type, as `Peers` is; it is ordinary GLP, in the repository.

```
person homophile.
Menu =::= homophile_menu.
procedure homophile_menu(Menu?, Person?, Integer?).
homophile_menu(M, _, S) :-
    random(S?, 100, K, S1), pick(K?, S1?, M?) @ 1/week.
procedure pick(Integer?, Integer?, Menu?).
pick(K, S, menu(Ss, _, same(P?))) :-
    K? =< 98, ground(Ss?) | one_of(Ss?, S?, P).
pick(K, S, menu(_, Os, other(P?))) :-
    K? > 98, ground(Os?) | one_of(Os?, S?, P).

person wary.
Approval =::= wary_approval.
procedure wary_approval(Approval?, Person?, Integer?).
wary_approval(C, _, S) :-
    random(S?, 100, K, _), answer(K?, C?) @ 1/day.
procedure answer(Integer?, Approval?).
answer(K, approval(_, yes)) :- K? =< 20 | true.
answer(K, approval(_, no))  :- K? > 20  | true.

run 100 agents
    [ approach ~ (homophile : 0.6 ; indifferent : 0.4),
      response ~ (wary : 0.7 ; sociable : 0.3) ]
    until 1 year seed 20260927.
```

A homophile answers a menu after a week on average, from when it is asked, and picks a peer of their own sex 98 times in 100, uniformly from the list the menu shows; the indifferent is the homophile at `2/week` and `50`. A wary person answers an offer after a day on average and declines four times in five; the sociable is the wary at `1/hour` and `90`. Approach and response make four profiles, and with sex eight kinds of agent. Each agent starts by asking its person the first menu.

[1] https://github.com/EShapiro2/GLP, directory `programs/sglp`.

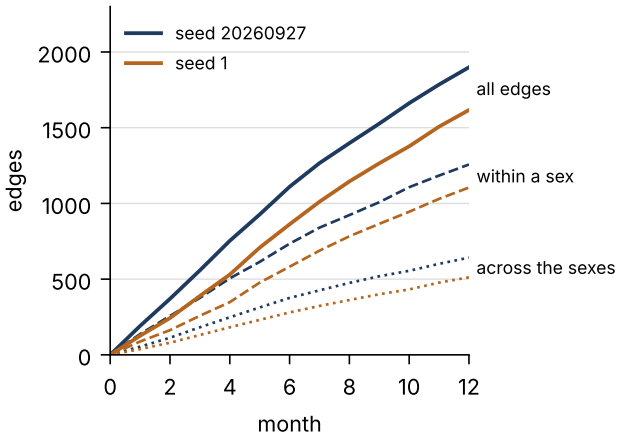


Figure 1: The edges of the friendship graph at the end of each month, in two runs of 100 agents for one year: all edges, those within a sex and those across the sexes.

Definition 5.1 (Friendship Graph of a Run). The *friendship graph* at simulated time t of a run of this population is the graph on its agents P whose edges are the $\{a, b\}$ for which the run's log (Definition 3.13) has an entry $(t', a, \texttt{approval(}b\texttt{, yes)})$ or an entry $(t', b, \texttt{approval(}a\texttt{, yes)})$ with $t' < t$.

An edge is an offer one of its two agents made and the other approved before t.

Results. We ran this population with 100 agents for one year, with the seeds 20260927 and 1, the first run taking 26 seconds of wall-clock time (Section 4); 58 agents drew homophile and 70 wary with the first seed, 60 and 77 with the second. With seed 20260927 the agents answered 7,532 menus, each answer an offer sent, and 7,523 offers they were asked to approve, 3,248 of them yes; with seed 1, 7,176 menus and 7,165 offers, 2,699 of them yes. Figure 1 shows the edges of the friendship graph at the end of each month. At the end of the year the graph has 1,899 edges with seed 20260927 and 1,617 with seed 1; it is one component from the fifth month on with seed 20260927 and from the fourth with seed 1, and at the end of the first month 7 agents have no friend yet with seed 20260927 and 10 with seed 1. An agent knows 13.8 other agents on average at the end of the first month, from 4 to 30, and 97.6 of the 99 at the end of the year, from 90 to 99, with seed 20260927; with seed 1, 8.8, from 4 to 19, and 96.7, from 84 to 99. Of the 2,450 pairs within a sex, 51% and 45% are friends by the end of the year, and of the 2,500 pairs across the sexes, 26% and 20%: the indifferent answer a menu twice as often as the homophile, so 31% and 30% of the offers cross the sexes, and the edges across the sexes are 34% and 32% of the edges. A yes to an offer from one who is already a friend adds no edge; with seed 20260927 that was 985 of the 2,242 yeses within a sex and 364 of the 1,006 across the sexes.

Table 1: The friendship graph at the end of the year by the mix of profiles, 100 agents, over the seeds 1 to 20: the edges and those across the sexes, mean ± standard deviation, and the month by whose end the graph is one component, mean and latest; *over the 19 runs in which it is one by the end of the year.

homophile	wary	edges	across the sexes	one by month
none	none	4,170 ± 30	2,101 ± 21	1.0, 1
none	70%	2,568 ± 151	1,291 ± 74	1.9, 3
none	all	1,538 ± 38	769 ± 26	2.5, 4
60%	none	3,128 ± 114	1,085 ± 104	1.2, 2
60%	70%	1,821 ± 157	590 ± 85	3.3, 6
60%	all	1,036 ± 64	328 ± 44	4.0, 6
all	none	2,108 ± 25	88 ± 12	1.3, 2
all	70%	1,221 ± 81	45 ± 6	3.7, 6*
all	all	668 ± 24	21 ± 5	4.9, 7

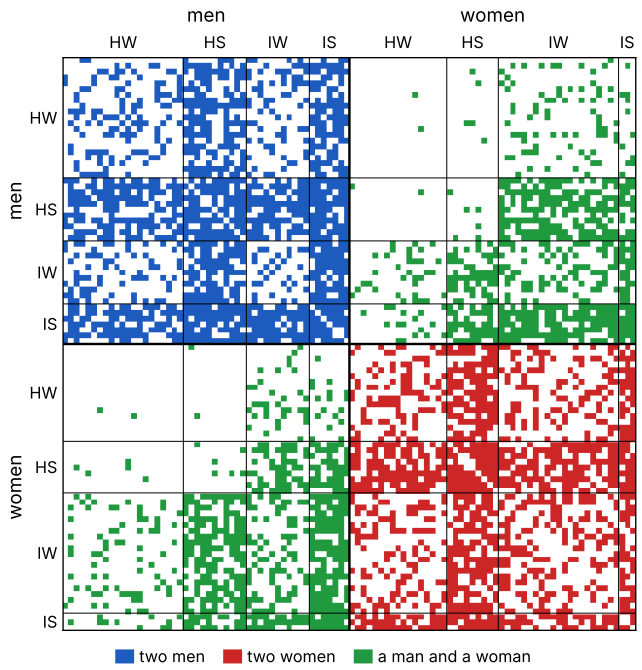


Figure 2: The friendship graph at the end of the year of the run with seed 20260927 as an adjacency matrix, a dot per edge: blue between two men, red between two women, green between a man and a woman. The agents are ordered by sex and within a sex by profile: homophile wary (HW), homophile sociable (HS), indifferent wary (IW), indifferent sociable (IS).

What the mix does. Table 1 has the friendship graph at the end of the year for each mix of none, 60% or all homophile with none, 70% or all wary, over twenty seeds, 1 to 20, the mean and the standard deviation of each count, and Figure 2 shows the graph of the run with seed 20260927 by profile: a sociable agent's row is dense, a wary one's sparse, and between two homophile of different sexes there is hardly an edge. The edges go from about 670 with all homophile and all wary to about 4,170 with none of either, the spread over the seeds at most 9% of the mean at every mix: the homophile offer half as often as the indifferent and to half the peers, and the wary accept a fifth of the offers where the sociable accept nine in ten, though many of the sociable's yeses go to friends already, so all sociable in place of all wary has four and a half times the yeses and about three times the edges. With all homophile the edges across the sexes are 3% to 4% of the edges and the graph within each sex is near complete, 2,020 edges of the 2,450 pairs with all sociable; with none homophile they are half the edges. In 179 of the 180 runs the graph is one component by the end of the year: by the end of the second month at every mix with none wary, and at the latest by the seventh; in the run with all homophile, 70% wary and seed 1 it is not one component by the end of the year.

Scale. The same population at 1,000 agents, 500 of each sex, seed 20260927, is 146,881 answers in one simulated year, in 167 minutes of wall-clock time (Section 4), 33,995 of the 73,372 offers answered yes. The friendship graph has 28,097 edges at the end of the year, 8,593 of them across the sexes; it is one component from the seventh month on, and at the end of the first month 62 agents have no friend yet.

6 EXAMPLE: COINS AMONG FRIENDS

The second platform is coins among friends, the coins-only restriction of grassroots currencies [29, 35]: a person's coins are units of their debt, a person holds only their friends' coins, and a swap at par between friends is mutual credit. The agent's question is a menu showing its friends and its wallet and taking a request, to pay a friend or to propose a swap to one; a friend's proposal is put to the person for approval, showing who offers how many and how many of that friend's coins the person holds, and the person answers yes or no. A payment sends the friend coins of their own, which they then no longer owe; an accepted swap issues to each party K coins of the other's; and while a proposal waits for the person's answer the agent takes neither a message nor a request. `Peer`, `Peers` and `YesNo` are those of Section 5.

```
Holding ::= holding(Peer, Integer).
Wallet  ::= List(Holding).
Request ::= pay(Peer, Integer) ; swap(Peer, Integer).
Menu    ::= menu(Peers, Wallet, Request?).
Offer   ::= offer(Peer, Integer, Integer, YesNo?).

%% Show the person their friends and wallet, and hand them
%% the request.
procedure (Menu)*ask(Peers?, Wallet?, Request).
(menu(Fs?, W?, R))*ask(Fs, W, R?).

%% Show the person who offers how many, and how many of
%% theirs they hold; hand them yes or no.
procedure (Offer)*respond(Peer?, Integer?, Integer?, YesNo).
(offer(From?, K?, Held?, A))*respond(From, K, Held, A?).
```

The agent's clauses, in the repository, send a friend the person pays what the wallet holds of that friend's coins, up to the amount asked, and ask again; send a friend the person chooses a proposal of K and ask again; put a proposal that arrives to the person, and on yes hold K of the proposer's coins and send them K of the agent's own, on no send a decline; hold coins that arrive from a friend, and hold nothing for a payment, which returns the agent's own coins; and drop the declines of their own proposals.

The profiles are two dimensions of two; one_of is that of Section 5:

```
%% Pay a friend whose coins I hold one to five; holding none,
%% propose a swap of ten to a friend.
procedure request(Integer?, Menu?).
request(S, menu(Fs, [], swap(F?, 10))) :-
    ground(Fs?) | one_of(Fs?, S?, F).
request(S, menu(_, [H|Hs], pay(F?, K?))) :-
    ground(H?), ground(Hs?) |
    random(S?, 5, K, S1), one_of([H?|Hs?], S1?, Hd),
    issuer(Hd?, F).

person spender.
Menu =::= spender_menu.
procedure spender_menu(Menu?, Person?, Integer?).
spender_menu(M, _, S) :- request(S?, M?) @ 1/day.

person cautious.
Offer =::= cautious_offer.
procedure cautious_offer(Offer?, Person?, Integer?).
cautious_offer(O, _, _) :- weigh(O?) @ 1/hour.
procedure weigh(Offer?).
weigh(offer(_, _, Held, yes)) :- Held? =< 20 | true.
weigh(offer(_, _, Held, no))  :- Held? > 20  | true.

run 100 agents
    [ spending ~ (spender : 0.5 ; saver : 0.5),
      credit ~ (cautious : 0.5 ; generous : 0.5) ]
    until 1 year seed 20260927.
```

A spender acts daily and a saver weekly, with the same `request`: each pays a friend whose coins they hold one to five, and holding none proposes a swap of ten to a friend drawn from the list; `issuer` reads the issuer of a holding. The cautious accept a proposal while holding at most twenty of the proposer's coins, and the generous accept one as it comes, each within an hour on average. Each agent is created with its identifier, its friends in the friendship graph at one year of the run of Section 5, and an empty wallet, and starts by asking its person the first menu, by its exported procedure `start`.

Definition 6.1 (Wallet, Circulation). Let $W_a(c)$ be the term at the argument of type `Wallet` of the goal of the procedure `agent` at agent a in a configuration c, the *wallet* of a in c. The *circulation* at simulated time t of a run r of this population is $\sum_{a \in P} \sum_{\texttt{holding(_, } k\texttt{)} \in W_a(c)} k$, c the last configuration of r with simulated time at most t.

The configuration c is quiescent, the clock advancing only at a Release (Definition 2.3), so every wallet in it is ground. Every holding is of another's coins, so the circulation is the debt the agents hold of one another.

Results. We ran this population with 100 agents for one year over the friendship graph at one year of Section 5's run of the same seed, with the seeds 20260927 and 1 (1,899 and 1,617 edges). With seed 20260927 the agents paid 17,334 times and proposed 2,929 swaps, 2,859 accepted and 70 declined; with seed 1, 18,222 pays and 3,121 proposals, 3,046 accepted and 75 declined. An accepted swap of ten issues twenty coins and a payment returns what it pays to its issuer, so the circulation is what has been issued less what has been paid: it is 2,987 and 2,830 coins at the end of the first month and grows by about 670 and 820 a month to 10,327 and 11,862 at the end of the year, held in 726 and 701 holdings by 96 of the 100 agents in both runs. The cautious decline a proposal from a friend of whose coins they hold more than twenty, so declines come as holdings grow: 70 of the 2,929 proposals were declined with seed 20260927, 65 of them in the second half of the year. Figure 3 shows who holds whose coins in the run with seed 20260927, in which 48 agents drew spender and 47 cautious: at the end of the year 9,685 of the 10,327 coins are spenders' coins in savers' wallets, the spenders having issued 10,061 of the coins out and the savers holding 9,742 of them, and the two run together from the first month; the other three pairs hold about 640 between them. Figure 4 shows the holdings one by one at the end of that year: the generous savers hold 6,738 coins and the cautious savers 3,004, where no cautious holding passes thirty, since the cautious decline above twenty; the cautious spenders have 5,376 coins out and the generous spenders 4,685; and the two spender profiles hold about 590 between them.

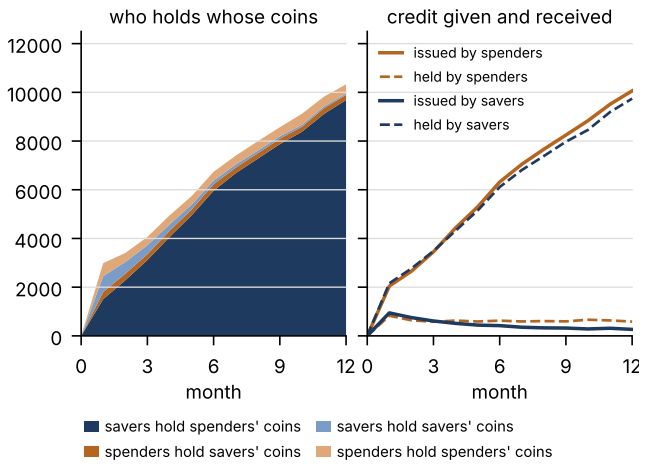


Figure 3: The run with seed 20260927, 100 agents, half spenders and half cautious, by month. Left: the circulation by who holds whose coins. Right: the coins issued by the spenders and by the savers, and those held by each — the credit each profile has given and received.

Table 2: Coins among friends at the end of the year by the mix of profiles, 100 agents over the friendship graphs of Section 5, over the seeds 1 to 20, mean ± standard deviation: the circulation, the proposals made and those declined.

spenders	cautious	circulation	proposals	declined
none	none	1,676 ± 108	693 ± 11	0
none	half	1,676 ± 108	693 ± 11	0
none	all	1,676 ± 108	693 ± 11	0
half	none	11,632 ± 652	2,959 ± 151	0
half	half	11,229 ± 609	2,999 ± 154	68 ± 19
half	all	10,072 ± 474	3,104 ± 174	242 ± 62
all	none	1,765 ± 173	4,384 ± 27	0
all	half	1,766 ± 173	4,383 ± 27	0.1 ± 0.2
all	all	1,765 ± 191	4,384 ± 27	1.8 ± 2.5

What the mix does. Table 2 has the circulation at the end of the year for each mix of none, half or all spenders with none, half or all cautious, over the seeds 1 to 20, each over the friendship graph

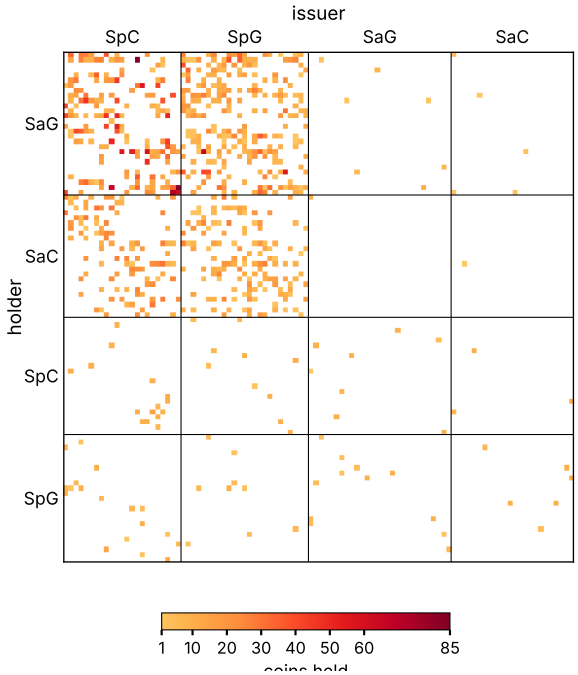


Figure 4: Who holds whose coins at the end of the year of the run with seed 20260927: holders by row, issuers by column, a cell the coins held, white where none. The agents are ordered by profile — spender (Sp) or saver (Sa), cautious (C) or generous (G) — the holders' profiles by the coins they hold and the issuers' by the coins they have out, most first.

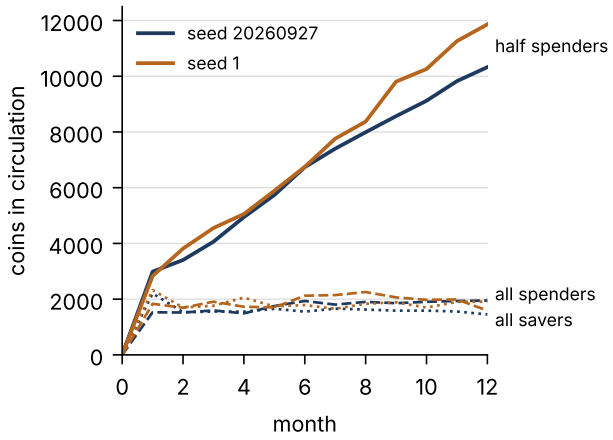


Figure 5: The circulation at the end of each month with half the agents cautious and none, half or all of them spenders, 100 agents for one year over the friendship graphs of Section 5, seeds 20260927 and 1.

of Section 5's run of the same seed, and Figure 5 the circulation by month with half cautious, for the two seeds of the figures. The circulation is highest where spenders and savers mix: about 11,200 coins at half and half, with a standard deviation of 610 over the seeds, and still growing, against about 1,700 to 1,800 with all savers or all spenders. A payment carries a friend's own coins back to them, and a spender pays daily, so with all spenders a swap's twenty coins are paid back within days and after about 4,380 swaps about 1,770 coins are out; a saver pays weekly, so with all savers few swaps are made and few coins are out; and at half and half a saver holds for weeks the coins of the spenders who pay it, which is where the circulation is. The credit dimension hardly bites at these holdings: with all savers the cautious decline no proposal and with all spenders at most nine in a run, so those runs differ little whatever the credit mix, and only with half the agents spenders do the cautious decline many, 68 of 2,999 proposals on average, and 242 of 3,104 with all cautious.

7 RELATED WORK

Six lines of work meet here: process calculi and languages with rates, simulated exactly; rule-based languages whose simulators do not enumerate the species they generate; languages and formalisms for continuous-time agent-based simulation; the agent-based platforms and the testing of multiagent systems in simulation; simulated people with designated profiles; and what the two examples study, homophily in the formation of friendships and the circulation of mutual credit. sGLP differs from all of them in one way: it simulates the program the platform deploys, and its stochastic specification covers the person. Appendix D expands each line.

Calculi and languages with rates. The stochastic pi-calculus attaches a rate to each prefix [23]; its biochemical variant moves the rate onto the channel and settles the race among the enabled communications by Gillespie's direct method [10, 24], and BioSPI implemented it in Flat Concurrent Prolog [24, 25, 27, 39], a monitor drawing the channel and advancing the clock, as GLP's engine does for sGLP. The stochastic pi-machine groups identical processes and counts the copies, and is proved reduction-equivalent to the calculus [22]; a later machine carries the grouping to restricted channels [21]. Bio-PEPA is a stochastic process algebra with the same simulation [5], and stochastic concurrent constraint programming attaches rates to a concurrent constraint language and simulates it by Gillespie's method [4], the nearest language to sGLP. The grouping is not available to us, since no two agents of a grassroots platform share a state; the proof we meet in the same form, Theorem 3.15.

Rule-based languages. Kappa's simulator does not count species at all [7]: it over-approximates the next events, rejects the draws that apply no rule, corrects the time advance, and wakes only the rules a static activation relation names; in sGLP the rules are the interactive types, few, and the instances the open questions, many, and the activation relation follows from the semantics, since a reduction reaches one agent.

Continuous-time agent-based simulation. ML3 specifies an agent by stochastic rules of a guard, a rate and an effect, alternative decisions competing by stochastic race, and simulates by the direct method with a dynamic dependency graph [14]; DEVS is the discrete-event formalism such simulators rest on [41]. ML3 is the nearest prior work to sGLP, and the difference is what is simulated:

there the rules are the model, here a vGLP program deployed as it stands, and the stochastic specification covers the person's answers.

Agent-based platforms, and testing in simulation. NetLogo, Repast, MASON, Mesa and krABMaga schedule every agent at each step of a discrete clock [1], where a stochastic race has no formulation [14], and none has a place for the person. Testing a multiagent system by simulating the code it deploys, with no surrogate, is argued for BDI agents mapped onto a discrete-event simulator [2]; there the agents are the system and no person is simulated, where here the program is the platform's and the person is what is drawn.

Simulated people with profiles. Generative agents give each simulated person a text persona and have a language model answer for them [20], OASIS runs a million such agents on a social platform [40], and user simulation evaluates interactive systems by simulated users with personas [3, 17]. An sGLP profile is a program with rates, typed by the dual of the question it answers, drawn from a seed and reproducible, and the simulation is a run of the deployed program; a persona is a text and its answers are a model's.

What the examples study. Homophily, the tendency to befriend one's like, is the best-attested regularity of social networks [19], and Currarini, Jackson and Pin model friendship formation under it, with the mixing set by a group's size and bias [6]; Section 5's runs are such a model's dynamics, with the bias a profile and the platform the one deployed. Mutual credit is studied on the Sardex network, its transactions' cycles and the liquidity they save [9, 13, 16], and modelled for LETS [26]; Section 6 simulates coins among friends over the friendship graph the social graph formed, with spending and credit as profiles.

The exponential assumption. The inter-event times of human activity are heavier-tailed than exponential; the non-Markovian and the Laplace Gillespie algorithms simulate renewal processes with other distributions [18], and nothing in sGLP fixes the exponential: drawing a rated goal's activation time from another distribution changes the draw and leaves the language as it is.

What is new. To the best of our knowledge, sGLP is the first language in which the person of a deployed multiagent program is specified as a stochastic agent answering the questions that program puts to them, so that one program is simulated at scale and deployed on a phone without being rewritten, and the first in which the stochastic specification is confined to the person while the machine keeps a nondeterministic semantics with no scheduling assumption.

8 CONCLUSION

sGLP is GLP with a rate on a goal, and a stochastic agent is an sGLP program in place of the UI of a vGLP platform and the person behind it, holding the person's end of each question the platform asks, typed by the dual of the question's interactive type. The platform's program is unchanged; a population is a census of persons, each with a role and attributes, and a mix of profiles for each role, each profile a program; and every simulated run is a run of the platform in which every answer was given by a stochastic agent, so what a simulation shows is something the platform can do.

Here we employed an exponential distribution; another distribution can be employed without changing the language [18].

The simulations done here show that platform behaviour can be tested and studied by mixing the stochastic profiles of its people: the friendship graph that forms, and how many coins circulate.

Four limitations. The simulator hosts the population in one machine, where the guard `when_idle` is sound (Section 4). A rate per act is a modelling assumption, and the inter-event times of people are heavier-tailed than exponential (Section 7). The profiles and their rates are written, not estimated from how people use the deployed platforms. Reading the logs and analysing the runs is software outside the language (Section 3).

REFERENCES

[1] Alessia Antelmi, Pasquale Caramante, Gennaro Cordasco, Giuseppe D'Ambrosio, Daniele De Vinco, Francesco Foglia, Luca Postiglione, and Carmine Spagnuolo. 2024. Reliable and Efficient Agent-Based Modeling and Simulation. *Journal of Artificial Societies and Social Simulation* 27, 2 (2024), 4. doi:10.18564/jasss.5300

[2] Martina Baiardi, Samuele Burattini, Giovanni Ciatto, and Danilo Pianini. 2026. Testing BDI-based Multi-Agent Systems using Discrete Event Simulation. *arXiv preprint arXiv:2602.13878* (2026). https://arxiv.org/abs/2602.13878

[3] Krisztian Balog and ChengXiang Zhai. 2024. User Simulation for Evaluating Information Access Systems. *Foundations and Trends in Information Retrieval* 18, 1–2 (2024), 1–261. doi:10.1561/1500000098

[4] Luca Bortolussi and Alberto Policriti. 2008. Modeling Biological Systems in Stochastic Concurrent Constraint Programming. *Constraints* 13, 1–2 (2008), 66–90. doi:10.1007/s10601-007-9034-8

[5] Federica Ciocchetta and Jane Hillston. 2009. Bio-PEPA: a Framework for the Modelling and Analysis of Biochemical Networks. *Theoretical Computer Science* 410, 33–34 (2009), 3065–3084.

[6] Sergio Currarini, Matthew O. Jackson, and Paolo Pin. 2009. An Economic Model of Friendship: Homophily, Minorities, and Segregation. *Econometrica* 77, 4 (2009), 1003–1045. doi:10.3982/ECTA7528

[7] Vincent Danos, Jérôme Feret, Walter Fontana, and Jean Krivine. 2007. Scalable Simulation of Cellular Signaling Networks. In *Proceedings of the 5th Asian Symposium on Programming Languages and Systems (APLAS'07)*. Springer, 139–157. doi:10.1007/978-3-540-76637-7_10

[8] Ohad Eitan, Idit Keidar, and Ehud Shapiro. 2026. The Grassroots Social Graph: A Secure Operating System for Multiagent, Distributed, Peer-to-Peer, Smartphone-Based Platforms. In preparation.

[9] Tomaž Fleischman, Paolo Dini, and Giuseppe Littera. 2020. Liquidity-Saving through Obligation-Clearing and Mutual Credit: An Effective Monetary Innovation for SMEs in Times of Crisis. *Journal of Risk and Financial Management* 13, 12 (2020), 295. doi:10.3390/jrfm13120295

[10] Daniel T. Gillespie. 1977. Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry* 81, 25 (1977), 2340–2361. doi:10.1021/j100540a008

[11] James Golike, Andy Lewis-Pye, and Ehud Shapiro. 2026. Formalising and Implementing Grassroots Social Contracts: From Legal Text to Grassroots Platforms (Full Version). *arXiv:2609.06231* (2026).

[12] Google. 2024. Dart Programming Language. https://dart.dev/

[13] George Iosifidis, Yanick Charette, Edoardo Airoldi, Giuseppe Littera, Leandros Tassiulas, and Nicholas Christakis. 2018. Cyclic Motifs in the Sardex Monetary Network. *Nature Human Behaviour* 2 (2018), 822–829. doi:10.1038/s41562-018-0450-0

[14] Till Köster, Oliver Reinhardt, Martin Hinsch, Jakub Bijak, and Adelinde M. Uhrmacher. 2024. A Fast Embedded Language for Continuous-Time Agent-Based Simulation. *Journal of Artificial Societies and Social Simulation* 27, 1 (2024), 10. doi:10.18564/jasss.5232

[15] Andy Lewis-Pye and Ehud Shapiro. 2026. Volition-Guarded Multiagent Atomic Transactions: Describing People and their Machines. *arXiv preprint arXiv:2604.25596* (2026). https://arxiv.org/abs/2604.25596

[16] Giuseppe Littera, Laura Sartori, Paolo Dini, and Panayotis Antoniadis. 2017. From an Idea to a Scalable Working Model: Merging Economic Benefits with Social Values in Sardex. *International Journal of Community Currency Research* 21 (2017), 6–21. doi:10.15133/j.ijccr.2017.002

[17] Yifan Simon Liu, Qianfeng Wen, Yilan Fan, Shirley Huang, Ruoqi Gao, Jianheng Hou, Muhammad Ahmed Mohsin, Zonglin Di, Brihi Joshi, Xincheng Tan, Yucheng Lu, Xiaoyi Liu, Heming Liu, Hanwen Xing, Guanghui Min, Zhengyang Shan, My Chiffon Nguyen, Ishan Gupta, Yunze Xiao, Hannah Collison, Jintao Huang, Jiatong Li, Sankalp Jajee, Yunhan Zhao, Bing Hu, Sky Ng, Xupeng Chen, Binghang Lu, Weihang Xiao, Aravind Mohan, Bolun Sun, Yunshu Wu, Yuanda Xu,

Yun Shen, Runyu Zhang, Zheyuan Deng, Zhiwei Zhang, Qianyu Zhu, Dianzhuo Wang, Yijun Wang, Yixuan He, Yuexing Hao, and Xiaomin Li. 2026. PersonaEval: Persona-Based User Simulation for Evaluating Interactive Applications. *arXiv preprint arXiv:2608.15838* (2026). https://arxiv.org/abs/2608.15838

[18] Naoki Masuda and Christian L. Vestergaard. 2021. Gillespie Algorithms for Stochastic Multiagent Dynamics in Populations and Networks. Elements in the Structure and Dynamics of Complex Networks, Cambridge University Press. arXiv:2112.05293

[19] Miller McPherson, Lynn Smith-Lovin, and James M. Cook. 2001. Birds of a Feather: Homophily in Social Networks. *Annual Review of Sociology* 27 (2001), 415–444. doi:10.1146/annurev.soc.27.1.415

[20] Joon Sung Park, Joseph C. O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST 2023)*. 1–22. doi:10.1145/3586183.3606763

[21] Loïc Paulevé, Simon Youssef, Matthew R. Lakin, and Andrew Phillips. 2010. A Generic Abstract Machine for Stochastic Process Calculi. In *Proceedings of the 8th International Conference on Computational Methods in Systems Biology (CMSB '10)*. ACM, 43–54. doi:10.1145/1839764.1839771

[22] Andrew Phillips and Luca Cardelli. 2007. Efficient, Correct Simulation of Biological Processes in the Stochastic Pi-calculus. In *Computational Methods in Systems Biology (CMSB 2007) (Lecture Notes in Bioinformatics, Vol. 4695)*. Springer, 184–199. doi:10.1007/978-3-540-75140-3_13

[23] Corrado Priami. 1995. Stochastic π-calculus. *Comput. J.* 38, 7 (1995), 578–589.

[24] Corrado Priami, Aviv Regev, Ehud Shapiro, and William Silverman. 2001. Application of a stochastic name-passing calculus to representation and simulation of molecular processes. *Information processing letters* 80, 1 (2001), 25–31. doi:10.1016/s0020-0190(01)00214-9

[25] Aviv Regev, William Silverman, and Ehud Shapiro. 2000. Representation and simulation of biochemical processes using the π-calculus process algebra. In *Biocomputing 2001*. World Scientific, 459–470. doi:10.1142/9789814447362_0045

[26] Jorim Schraven. 2001. Mutual Credit Systems and the Commons Problem: Why Community Currency Systems such as LETS Need Not Collapse Under Opportunistic Behaviour. *International Journal of Community Currency Research* 5, 4 (2001). doi:10.15133/j.ijccr.2001.002

[27] Ehud Shapiro. 1989. The family of concurrent logic programming languages. *ACM Computing Surveys (CSUR)* 21, 3 (1989), 413–510. doi:10.1145/72551.72555

[28] Ehud Shapiro. 2021. Multiagent Transition Systems for Composing Fault-Resilient Protocol Stacks. *arXiv preprint arXiv:2112.13650* (2021). https://arxiv.org/abs/2112.13650

[29] Ehud Shapiro. 2022. Grassroots Currencies: Turning Mutual Trust into Liquidity. *arXiv preprint arXiv:2202.05619* (2022). https://arxiv.org/abs/2202.05619

[30] Ehud Shapiro. 2023. Grassroots Distributed Systems: Concept, Examples, Implementation and Applications (Brief Announcement). In *37th International Symposium on Distributed Computing (DISC 2023). (Extended version: arXiv:2301.04391)*. LIPICS, Italy, 47:1–47:7. https://arxiv.org/abs/2301.04391

[31] Ehud Shapiro. 2023. Grassroots Social Networking: Serverless, Permissionless Protocols for Twitter/LinkedIn/WhatsApp. In *OASIS '23* (Rome, Italy). Association for Computing Machinery. doi:10.1145/3599696.3612898

[32] Ehud Shapiro. 2024. A Grassroots Architecture to Supplant Global Digital Platforms by a Global Digital Democracy. *arXiv:2404.13468, Proceedings of DAWO'24* (2024). https://arxiv.org/abs/2404.13468

[33] Ehud Shapiro. 2025. Characterising Global Platforms: Centralised, Decentralised, Federated, and Grassroots. *arXiv preprint arXiv:2511.03286* (2025). https://arxiv.org/abs/2511.03286

[34] Ehud Shapiro. 2025. GLP: A Grassroots, Multiagent, Concurrent, Logic Programming Language for AI (full version). *arXiv preprint arXiv:2510.15747, Summary in Proc. of ICLP'26, EPTCS 450, pp. 119–133* (2025). https://arxiv.org/abs/2510.15747

[35] Ehud Shapiro. 2026. Grassroots Bonds: Financing by the People, for the People. arXiv:2603.13671. arXiv:2603.13671 https://arxiv.org/abs/2603.13671

[36] Ehud Shapiro. 2026. Implementing Grassroots Logic Programs with Multiagent Transition Systems and AI (Full Version). *arXiv:2602.06934. Summary to appear in Proc. of LOPSTR+PPDP'26* (2026). https://arxiv.org/abs/2602.06934

[37] Ehud Shapiro. 2026. Types for Grassroots Logic Programs (Full version). *arXiv preprint arXiv:2601.17957* (2026). https://arxiv.org/abs/2601.17957

[38] Ehud Shapiro. 2026. Volition Elicitation: Operational Semantics for People and Their Machines. arXiv:2607.14138. arXiv:2607.14138 [cs.DC] https://arxiv.org/abs/2607.14138

[39] William Silverman, Michael Hirsch, Avshalom Houri, and Ehud Shapiro. 1988. The Logix system user manual Version 1.21. In *Concurrent Prolog: Collected Papers*. MIT Press, 46–77.

[40] Ziyi Yang, Zaibin Zhang, Zirui Zheng, Yuxian Jiang, Ziyue Gan, Zhiyu Wang, Zijian Ling, Jinsong Chen, Martz Ma, Bowen Dong, Prateek Gupta, Shuyue Hu, Zhenfei Yin, Guohao Li, Xu Jia, Lijun Wang, Bernard Ghanem, Huchuan Lu, Chaochao Lu, Wanli Ouyang, Yu Qiao, Philip Torr, and Jing Shao. 2024. OASIS: Open Agent Social Interaction Simulations with One Million Agents. *arXiv preprint arXiv:2411.11581* (2024). https://arxiv.org/abs/2411.11581

[41] Bernard P. Zeigler, Herbert Praehofer, and Tag Gon Kim. 2000. *Theory of Modeling and Simulation: Integrating Discrete Event and Continuous Complex Dynamic Systems* (2nd ed.). Academic Press.

A PROOFS

Let r be an sGLP run of the simulation program N of (M, P, π) over P from its initial configuration c^0, π over a well-formed population, and σ the simulation mapping of r.

Theorem 3.15 (Soundness, restated). *$\sigma(r)$ is a proper safe computation of the vmaGLP transition system over P and M from $\sigma(c^0)$ in which every Answer is the image of a Reduce of a person goal.*

Proof. Let $r = (c^1, t_1, \alpha_1), (c^2, t_2, \alpha_2), \ldots$ with $(c^1, t_1, \alpha_1) = (c^0, 0, \emptyset)$. Throughout, a variable of I_a occurs nowhere before the transition that introduces it, by the properness of the maGLP computation $c^1, c^2, \ldots$ (Definition 2.4); every clause of N is SRSW (Definition E.4); a Communicate removes the assignment it delivers from the readers substitution of its agent (Definition E.7), and σ_r keeps one delivered to a person goal until the reader it assigns is presented (Definition 3.14); and a person goal holds a variable of I_a only as the person process holding an ask, as a relay or a goal descended from one, or as a screen holding the program's output, since the person process passes the person's end of each ask to the relay it spawns, a relay puts no variable it receives into the copy (Definition 3.7), and a screen assigns the writer of a request a list of the elements it has shown, each ground (Definition 3.8), so that the goals descended from the person procedures' goals, and the `merge` and `random` goals the person process spawns, hold the copies' variables, the person, screen streams, seeds and ground lists, and none of I_a.

Safety. We show that each transition $(c^i, t_i, \alpha_i) \rightarrow (c^{i+1}, t_{i+1}, \alpha_{i+1})$ of r maps under σ to a transaction of the vmaGLP transition system over P and M or to a stutter; then $\sigma(r)$ is a safe computation from $\sigma(c^0)$ (Definitions E.1 and E.2). A Release has $c^{i+1} = c^i$: a stutter. A Machine transition is a Reduce, a Communicate or a Cold-call of the maGLP transition system over P and N (Definition 2.3), and we take each by what it reduces or delivers.

(1) A Reduce at a of a platform goal with an asking clause of $\lceil M \rceil$ replaces $q(S_1, \ldots, S_n, D)$ by $q_1(S'_1, \ldots, S'_n, X?, D')$ or $q_1(S'_1, \ldots, S'_n, X, D')$, X fresh and of $\mathcal{V}_a$, the clause being renamed into $\hat{\mathcal{V}}_a$ (Definition E.7), and adds to σ_a the assignment of D to the list of the ask `ask(`T`, `$t(E)$`)` and the tail $D'?$, E the person's end of X, and the assignments by which each S'_l passes its argument on (Appendix E, the asking clause). In the image, G loses $q(S_1, \ldots, S_n)$ and gains the asked goal, read through the pairs of its arguments and with the identifier of the goal reduced; H gains E, a variable of I_a that now occurs in an ask in σ_a; σ_r is unchanged, D being an ask-stream variable, the assignments of the pairs being outside it, and the assignment $W? := S_l?$ of an output position, read through, being absent while $S_l?$ is unassigned (Definition 3.14). This is the Ask of a's goal with interactive variable X (Definition E.14).

(2) A Reduce at a of a platform goal with a compiled clause of M: G changes as the Reduce of the source goal with the source clause does (Lemma E.24), the platform goals and clauses of N being those of $\lceil M \rceil$ and read back as there; σ_r gains the assignments the source reduction adds, read through the pairs as there, an assignment to a variable of I_a at a position the program writes among them where the clause writes the program's output to the person; and H is unchanged, since the reduction touches no person goal and no ask, a variable it introduces occurs in neither, and a reader of I_a whose writer it assigns stays in H as itself, a relay presenting it only once the assignment is delivered to it (Definition 3.7 (*ii*)). This is the source Reduce. A Reduce of a platform goal with a `merge` clause the compilation adds, which moves asks between ask streams, changes nothing σ reads: a stutter.

(3) A Reduce at a of a person goal that adds no answer to a variable of I_a and presents no reader — the person process, which in reader mode gives the place of the writer an ask carries to another, a screen, which shows an element or assigns the writer of a request the elements it has shown, a goal descended from the person process other than a relay, or a relay or a goal descended from one that gives a writer's place to another, discards a writer or assigns nothing received: G and σ_r are unchanged, a writer that takes another's place being read as its question; and so is H: a variable of I_a in the goal reduced passes to a goal the reduction spawns, directly or by an assignment it adds for a reader that goal holds, by SRSW for the person process, whose clause passes the person's end of an ask, or the writer that takes its place, to the relay it spawns, by Definition 3.7 for a relay, a writer it discards being in H before, in the goal reduced, and after, discarded (Definition 3.14), and by Definition 3.8 for a screen, which passes the rest of the output it holds to the screen it spawns and none of whose variables H counts (Definition 3.14), and any other person goal holds none; a variable of I_a the reduction introduces takes the place of a writer the goal held, and is read as its question; an unpresented reader of I_a at a position the program writes is in H as itself before and after, and the variables of its contents are in H neither before nor after (Definition 3.14); and the readers presented are those presented before. A stutter.

(4) A Reduce at a of a relay or a goal descended from one that presents a reader $X?$ of I_a, at one place and answering nothing (Definition 3.7 (*ii*)). $X?$ is at a position the program writes and is in the contents of no other unpresented reader, the copy at the place of an unpresented reader having no place inside it. $X?$ was assigned a term u at some agent b, and $\hat{u}$, u with each reader replaced, repeatedly, by the term assigned to it, has no reader and is the term presented, its writers being at the positions the person writes; each assignment used in forming $\hat{u}$ was delivered to the relay or a goal descended from it and is not presented (Definition 3.7 (*ii*)), so it is in σ_r of its agent, being added by a Reduce of a platform goal or an answer, or it is one by which a writer takes another's place or by which an asking clause passes an argument on, which σ reads through. Before, $X? \in H$ and none of the variables of its contents, with the assignment to $X?$ in σ_r of b, read through, and $\hat{u}$ with no reader in $\sigma(c^i)$; after, $X?$ and the readers replaced in forming $\hat{u}$ are presented, so H loses $X?$ and gains the variables of I_a the relay holds of $\hat{u}$, read as in G, each a writer at a position the person writes and so a question, and the assignment of $X?$ with $\hat{u}$, shown; σ_r of each agent loses the assignments used in forming $\hat{u}$, their readers being presented; and G is unchanged, the copy's variables being outside I_a. This is the Communicate of the assignment to $X?$ in σ_r of b to a's person, delivering $\hat{u}$ with the assignments used in forming it, which it removes (Definition E.13).

(5) A Reduce at a of a person goal that adds an answer $X? := u$ to a writer $X \in I_a$. The goal is a relay or a goal descended from one, the person process giving a writer's place to another and adding no answer, and by Definition 3.7 the assignment is the one

answer of the reduction, X a writer at a position the person writes that the relay received or that takes the place of one, read as its question, and u is a term of the type of X, a list cell where X is of a stream type and a list cell or `[]` where it is of a list type, ground but for fresh variables of $\hat{\mathcal{V}}_a$, the relay's clause being renamed into it: readers $Y_1?, \ldots, Y_k?$ at the positions the person writes of type `Stream(`T'`)` or `List(`T'`)`, but where the copy ends the list there with `[]`, and writers $W_1, \ldots, W_m$ at the outermost positions the program writes, $Y_1, \ldots, Y_k$ and $W_1?, \ldots, W_m?$ passed on to goals of the relay. $X \in H$ before, occurring in the goal reduced, and not after, since X occurs only in the goal reduced, which assigns it, and each writer whose place X takes is already assigned, so that no writer read as the question of X occurs after; the variables of u are in I_a (Definition 3.12), and the readers $W_j?$ are at positions the program writes, received through the relay's R, not presented, and in the contents of no unpresented reader, nor is X, a writer whose place in the copy the relay answers from, so $H' = H \setminus \{X\} \cup \{Y_1, \ldots, Y_k, W_1?, \ldots, W_m?\}$; σ_r gains $X? := u$; and G is unchanged. The reader of X is reachable in c^i (Definition 3.7 (*i*)), so the reader of the question of X occurs in $\sigma(c^i)$: a goal that holds the reader of a writer of I_a at a position the person writes is a platform goal, which G reads, or a person goal that holds it in a term of an assignment σ_r keeps, its reader not presented; and an assignment whose term holds one is of σ_r, or is one by which a writer takes another's place or by which an asking clause passes an argument on, which σ reads through. So X is open in $\sigma(c^i)$ (Definition E.18), and this is the Answer of a to X with u (Definition E.15).

(6) A Communicate at a delivering an assignment to an ask-stream variable to a `merge` goal the compilation adds or to the person process, or delivering to a person goal other than a screen an assignment other than an answer that a Reduce of a person goal added: the variables of I_a it moves are in H before and after, or, in the contents of an unpresented reader, in neither, and G and σ_r are unchanged: a stutter. A Communicate, within a or from another agent, delivering to a relay of a or a goal descended from one an assignment to a reader received through its R, a reader at a position the program writes: the reader is not yet presented, a relay presenting a reader only once its assignment is delivered (Definition 3.7 (*ii*)), so H holds as itself before and after the reader, or the unpresented reader in whose contents it is, and none of the variables of its contents; σ_r keeps the assignment, delivered to a person goal and its reader not presented; and G is unchanged: a stutter. A Communicate of an assignment an asking clause's reduction made, or of one by which a writer takes another's place, replaces a variable by what σ already read it as: a stutter. A Communicate delivering to a platform goal other than a `merge` goal the compilation adds an assignment added by a Reduce of a platform goal, within a or from another agent, or an answer that a relay of an agent b added to a writer of I_b: the source Communicate, G changing as there, σ_r losing the assignment as there, and H not at all. A Communicate delivering to a person goal an assignment of a variable that is not in I_a, which is in no σ_r, or any assignment to a screen, which contributes nothing to G or H and whose reader is never presented, so that σ_r keeps the assignment (Definition 3.14): a stutter.

(7) A Cold-call from a to b moves a message between the network streams, which are arguments of platform goals: the source Cold-call.

Every Answer is the image of a Reduce of a person goal: by the cases, each is the image of a Reduce of a relay or a goal descended from one, of case (5), and of no other.

Properness. Let a variable occur in the source goals of a in $\sigma(c^{i+1})$ and not in $\sigma(c^i)$. Read as in G, it occurs in a platform goal of a in c^{i+1} other than a `merge` goal the compilation adds, and in no goal of a in c^i: a variable of such a goal in c^i is one of its source, read as in G; the variable of an ask reaches only `merge` goals, the person process and a relay; and a variable a person goal holds reaches no platform goal, the variables of an answer being fresh, a writer that takes another's place being read as its question, and a screen assigning only the writers of requests, with ground lists. So it occurs in no goal of a earlier, by the properness of the maGLP computation, hence in no H earlier, the variables of H occurring, in that configuration or an earlier one, in goals of a or in terms of σ_a assigned to ask-stream variables or to readers that person goals of a hold, and those of a shown assignment not being occurrences (Definition E.12). The clause each source Reduce renames is, but for its ask streams, the compiled clause that the Reduce of r it is the image of renames apart from every agent's goals and readers substitution in c^i and in every earlier configuration (Definitions 2.4 and E.17), so from every agent's source goals, held variables and readers substitution in $\sigma(c^i)$ and earlier. Hence $\sigma(r)$ is proper (Definition E.17, read for a computation). □

Corollary 3.16 (restated). *Where every g_a is the initial goal* `agent(ch(_?,_),ch(_?,_))` *of M, with O_a one of its writers where M defines* `Output`*, $\sigma(r)$ is a proper safe run of the vmaGLP transition system over P and M.*

Proof. In c^0 agent a holds the platform goal g'_a and the person goals $person_{\pi(a)}(U_a?, h_a, R_a, s_a)$ and $screen(O_a?, [], R_a?)$, which hold no variable of I_a that H counts, H excluding the screen, and $\sigma_a = \emptyset$; where M defines `Output`, $O_a?$ occurs in c^0 only in the screen, which σ drops, so O_a occurs once in $\sigma(c^0)$, as a writer of g_a does; so $\sigma(c^0)$ has at each a the agent state $(\emptyset, \{g_a\}, \emptyset)$, which is the initial local state of the vmaGLP transition system over P and M where g_a is `agent(ch(_?,_),ch(_?,_))` (Definition E.16); so $\sigma(c^0)$ is its initial configuration, and $\sigma(r)$, a proper safe computation from it by Theorem 3.15, is a proper safe run (Definition E.1). □

B IMPLEMENTATION NOTES

The decisions the implementation takes where Sections 2, 3 and 4 leave a choice open, recorded here so that a reader of the code finds them stated. The code is `programs/sglp` of the GLP repository: `monitor.glp`; `sglp.glp`, the types and procedures that the transformation, the programs it prints and their harnesses share — the units of a horizon, the slot in which a rated goal receives the writer of its answer, and the term form; `transform/transform.glp`, the transformation, and `transform.sh`, which runs it; and, in `examples/`, a directory per example with the vGLP program (`graph.vglp`, `coins.vglp`), its canonical compilation by hand, `graph.glp` or `coins.glp`, clause for clause as the README records, its sGLP source in the term form (`graph_sglp.glp`, `coins_sglp.glp`), the

two modules the transformation prints from it — `profiles.glp` for the profiles and `population.glp` for the population — its harness, its run script and its reader of the log; and the tests, `test_sglp.sh` and `tests/`.

- The clock is in simulated seconds; a year is 365.25 days, and the units of `until` and of a rate are second, minute, hour, day, week and year, singular or plural after `until`. The horizon is inclusive: the clock never passes it, and the monitor stops when the least activation time would, binding every pending token to `stop`, which ends the rated goal waiting on it, or when the queue is empty at rest, and writes `clock(T)` last.
- The queue, the draws and the clock are the engine's (the GLP paper's appendix on simulated time [34]): the transformation converts a rate of `N` per unit to `N` over the unit's seconds, a number a second, and the horizon to seconds; two tokens of one activation time are released in the order they were made pending; the clock's seed is the one the monitor gives `start_clock`.
- Seeds: every seed is split from the run's seed by `split_random_seed`, in a fixed order: the clock's, then the census's where the run names one, then one for each agent in the order of the persons, from which are drawn, in order, its profile per dimension of its role, in the order of the run declaration's mix, and then split the seed of its person process and, in the social graph, the seed from which the two agents of its own sex it knows at the start and then the two of the other sex are drawn, each pair uniformly and without replacement; the person process splits for each person goal a seed and continues with the other. Each run's draw — how many agents of each profile, and which agent has which — is computed again from the seed, without the run, and written beside the log.
- The agents are numbered $1, \ldots, n$, n even and at least 6, so that each agent of the social graph can know two others of its sex at the start, both its lists being non-empty, as `Peers` requires; in the social graph the first half are of one sex and the rest of the other, and each agent's lists are kept in increasing order, without repetition, as people are added to them. Where the run declaration gives a number n of agents, agent a's key is a, and on a canonical platform of vGLP [38], whose keys are strings, the letter `p` followed by a padded with zeros to the width of n; where it names a census, agent a's key is that of the a-th person the census draws. Placement is attribution: an agent's goals are its `start` goal, its person process, its screen and what they spawn; the harness's goals belong to no agent.
- The person process reads the ask stream of the agent's initial goal, in the dispatcher's place, and its answer and present procedures are the relay of each question (Definition 3.7): a person goal answers into its copy of the question, and the answer procedure hands the program's variable the answer once the positions the person writes are ground but for those of a stream or a list, which it hands over one element at a time, the outermost positions the program writes as fresh writers whose readers the person process keeps, a step after the person goal's; the program's term reaches the person goal the same way, by the present procedures, once the positions the program writes are ground, the person's positions inside it open; and a term crossing at any depth, a question inside an answer included, crosses the same way, so that every answer is logged. Two positions the person writes in one alternative are two questions, each with its own answer procedure and log line, and a list the program writes inside a question is shown whole, once its last cell is written; and the person process and the screen ask the monitor for the clock by a request `time(T)`, answered at once, to log it.
- The monitor's input, the input of each screen and the log's sink are mutual references. Each is allocated by the root's `allocate_mutual_reference/2`, and `stream_append/3` writes to it; the network is the root's `mwm/2` over the agents' output streams and a balanced tree of `deliver/4` goals. The log's sink is read by `send_to_user/1` and written tab-separated by the harness.
- `one_of` and the `nth` it calls take a non-empty list, of the type `NonEmpty(X) ::= [X | List(X)]`, and each has a clause for every list of that type, as the type checker requires of every procedure [37].
- The log is one line per answer: the simulated time in seconds, the agent, the interactive type and the term handed over, the positions the program writes as variables; and one line per element of the output shown: the simulated time, the agent, `output` and the element; the fields separated by tabs.
- The transformation prints one term a line, operators prefix, and chooses a question's person procedure by the functor of its term, so the interactive types of a program must have distinct functors, which it checks; each dimension's draw is printed with the population. `one_of` is shared by the two programs.
- Coins among friends: the messages are `pay`, `coins`, `propose` and `decline`. A wallet keeps its holdings in order, a new issuer's at the end, and a holding that reaches zero is removed; the order decides only which holding a request draws. A pay of a friend whose coins the wallet does not hold would send zero coins and change nothing, and does not arise, since a request pays an issuer its menu shows. While a proposal waits for the person's answer the agent takes neither a message nor a request; after a yes the wallet is rebuilt before either is taken, and a request answered while a proposal waited is acted on after the proposal; after a pay the agent asks again once the wallet after the pay is known. The harness takes each agent's friends from the friendship graph of Section 5's run of the same seed and refuses a graph on which an agent has no friend, an agent's friends being of the non-empty type `Peers`. The circulation of Section 6 is replayed from the log by the agent's clauses, a request answered while a proposal waits being acted on after the proposal, and the holdings of its figures with each move at its answer; in the run with seed 20260927 the two agree at the end of the year, 10,327 coins.

C THE MONITOR

The monitor steps GLP's simulated clock [34], which implements Gillespie's algorithm [10] in the first reaction form of Section 2, over the guard `when_idle`. This is the whole module as it is in the repository, `programs/sglp/monitor.glp`, its comments replaced by this text; the transformation, the two examples, their harnesses and their tests are in the same directory.

GLP's types are `Token ::= go(Number) ; stop`, the token a rated goal waits on, and `ClockStep ::= released(Number) ; stopped(Number)`, what a step of the clock reports. A request to the monitor is `time(T)`, answered at once with the clock of the last step.

```
Request ::= time(Number?).
Clock   ::= clock(Number).
```

The monitor goal holds its input stream, the clock of the last step in seconds, the horizon in seconds and the log's reference; it starts the engine's clock with the run's seed for it, and its own at zero.

```
exported procedure monitor(Stream(Request)?, Integer?, Number?,
                           MutualRef?).
monitor(In, Seed, Horizon, Log) :-
    start_clock(Seed?),
    monitor_loop(In?, 0, Horizon?, Log?).

procedure monitor_loop(Stream(Request)?, Number?, Number?, MutualRef?).
monitor_loop([time(Clock?)|In], Clock, Horizon, Log) :-
    monitor_loop(In?, Clock?, Horizon?, Log?).
monitor_loop(In, _, Horizon, Log) :-
    when_idle |
    step_clock(Horizon?, Step),
    stepped(Step?, In?, Horizon?, Log?).

procedure stepped(ClockStep?, Stream(Request)?, Number?, MutualRef?).
stepped(released(T), In, Horizon, Log) :-
    monitor_loop(In?, T?, Horizon?, Log?).
stepped(stopped(Clock), _, _, Log) :-
    stream_append(clock(Clock?), Log?, _).
```

A `time` request is answered from the clock of the last step. The step clause is taken only when the machine rests, so every rated goal spawned has made its token pending and the queue is that of the configuration: `step_clock` moves the clock to the least activation time within the horizon and binds that token to `go(T)`, which is Release, and the monitor goes on with `T` as its clock; when the least activation time is past the horizon, or no token is pending, `step_clock` binds every pending token to `stop`, which ends each rated goal still waiting, and the monitor writes the clock to the log and ends. The clauses are tried in order, so a request that has arrived is answered before the machine is found idle.

D RELATED WORK, CONTINUED

Section 7's six lines of work, each expanded.

Calculi and languages with rates. BioSPI implemented the biochemical stochastic pi-calculus in Flat Concurrent Prolog [27]: each channel is an object carrying a base rate, processes post send, receive, dimer and withdraw requests to it, a monitor and the channel objects jointly compute the weighted rates, and the monitor draws the channel, completes one transmission, withdraws the losing offers of the two chosen processes and advances a clock [24, 25, 39]. Bio-PEPA [5] is a process algebra for biochemical networks in the PEPA family, with stochastic simulation by Gillespie's method among its analyses; stochastic concurrent constraint programming [4] attaches a rate to each action of a concurrent constraint program, with a continuous-time Markov chain semantics simulated by Gillespie's method, which makes it the nearest language to sGLP: a concurrent constraint language with rates. The difference is where the rate sits and what is simulated: in sCCP every action is rated and the program is the model, in sGLP the platform's program keeps vGLP's semantics unchanged and only the person's goals are rated.

Two criticisms of BioSPI's implementation. The stochastic pi-machine makes the cost of a simulation depend on the number of species rather than the number of molecules, by grouping identical processes and counting the copies, and reports its gene example running in ten seconds where their own previous version of SPiM took eight minutes and BioSPI twenty-one; the criticism is that the earlier simulators, BioSPI among them, describe no algorithm for keeping track of identical processes, the separate process per molecule being what they report of their previous version [22]. A later machine for stochastic process calculi carries the same point to restricted channels, and against the stochastic pi-machine itself: it stores each complex as one species, where a simulator in which every restricted channel has a globally fresh name carries one species and one reaction per copy [21]. The second criticism is of a different kind: the stochastic pi-machine is proved reduction-equivalent to the calculus and shown to preserve apparent rates, which they report as the first correctness proof for the stochastic pi-calculus to cover optimisations of this kind; their own earlier machine was proved correct against a variant of the calculus [22].

What sGLP does about them. The first economy is not available to us, and not because of how we implement: two agents are interchangeable only while their machine states agree, and in a grassroots platform they stop agreeing as soon as they have different friends, so a population of a thousand people is a thousand states, no two with the same open questions. What carries over is the rest of the machinery — the next of M pending goals is found in $O(\log M)$ by a priority queue, where the linear scan of the direct method costs $O(M)$ and a tree of propensities $O(\log M)$ [14, 18] — together with one thing a vGLP program has for nothing: a reduction changes the resolvent of one agent, so an event touches the questions of that agent and of no other. Their grouping also needs every choice to carry an identifier, and where a choice has none their encoding creates an extra definition for it; a person declaration names an interactive type, so we have the identifier already [22]. They also record a restricted channel's apparent rate as a delay, to keep the machine compositional where several machines run in parallel, on distributed or multi-core systems [22]; that is the setting of a deployed platform, one machine per agent, and not yet of our simulator, which hosts the population in one machine (Section 4). The second criticism we accept as the standard to meet, and meet it in the same form: the runs of the simulator are runs of the vmaGLP transition system of the very program being simulated, with the person's answers drawn rather than given.

Rule-based languages. Kappa's simulator [7] keeps a superset of the events that may happen next, rejects the draws that do not correspond to a rule application, corrects the time advance so that the timings stay exact, and wakes only the rules a static activation relation names; its cost per event is independent of the number of species and of the number of agents, and logarithmic in the number

of rules. In sGLP the rules are the interactive types, of which a platform has few, and the instances are the questions open at the agents, of which a population has many; the activation relation it computes statically we have from the semantics, since a reduction reaches one agent.

Continuous-time agent-based simulation. In ML3 [14] agents cannot be grouped into subpopulations, so handling the transitions efficiently is what decides the cost, and the dependency graph is dynamic where a biochemical one is static. DEVS [41] is the formalism of discrete-event simulation — atomic models with time advances, coupled into networks — on which ML3 and the discrete-event simulators of agents rest; sGLP's Release is a discrete event in that sense, and its machine steps between events take no time.

Agent-based platforms, and testing in simulation. In NetLogo, Repast, MASON, Mesa and krABMaga [1], and in agent-based modelling at large, the clock is a discrete step: the choice of step size bears on the validity of the results, and an alternative that is retracted when another decision wins — a stochastic race — has no formulation [14]. A platform's agent is the whole of what is modelled, where a vGLP agent is a person and a machine, and only the person is drawn. Baiardi, Burattini, Ciatto and Pianini [2] argue that a multiagent system should be tested in simulation on the very specification that is deployed, with no surrogate, and map the control flow of BDI agents onto a discrete-event simulator at several granularities; the demand is ours too, and what is tested differs: their agents are the system under test and their environment is the simulator's, where in sGLP the system under test is a vGLP program whose agents are people and machines, and the simulation replaces the people.

Simulated people with profiles. Generative agents [20] are simulated people whose memory, reflection and planning are done by a language model from a persona written as text, in a sandbox town; OASIS [40] scales such agents to a million on a simulated social platform, with recommendation and the platform's actions modelled; user simulation in information retrieval [3] evaluates search and recommendation systems by simulated users, and PersonaEval [17] evaluates interactive applications by persona-based simulated users driven by language models. All simulate the person by a persona and a model; none runs the deployed program, and none specifies the person's behaviour stochastically, with a stated law. An sGLP profile is the person's behaviour as a program with rates, typed by the dual of the question it answers, so that the simulation is a run of the deployed program (Theorem 3.15), drawn from a seed and reproducible; it lacks the persona's richness, and the rates and the profiles are to be estimated from how people use the deployed platforms.

What the examples study. McPherson, Smith-Lovin and Cook review homophily across the dimensions of social life, sex, age, race, religion, education and the rest, and find it structuring every kind of tie [19]. In Currarini, Jackson and Pin's model agents of several types form friendships by meeting, with a preference bias for their own type, and they derive how the share of same-type friendships and the number of friends vary with a group's relative size and its bias, which they fit to the Add Health data [6]; sGLP's homophile and indifferent profiles are the bias, and Table 1 and Figure 2 are what the deployed platform does under it, not a model of it. On mutual credit, Iosifidis et al. analyse the cyclic motifs of the Sardex transaction network, where credit returns to its issuer around cycles [13]; Fleischman, Dini and Littera measure the liquidity obligation-clearing and mutual credit save on that network's data [9]; Littera et al. describe the system's design and history [16]; and Schraven models why a LETS need not collapse under opportunism [26]. None simulates the deployed program; Section 6's runs are of coins among friends as the platform's program executes it, with the people drawn.

The exponential assumption. The non-Markovian Gillespie algorithm simulates coupled renewal processes by a first-order expansion accurate for large numbers of processes, and the Laplace Gillespie algorithm simulates exactly those whose inter-event distribution is a mixture of exponentials, which covers the fat-tailed distributions common in human interaction [18]; a rate per act is a modelling assumption and not a fact about people.

E DEFINITIONS AND RESULTS RECALLED FROM VGLP

Sections 2 and 3 and Theorem 3.15 rest on the following definitions and results of vGLP [38], which rest in turn on GLP [34] and its types [37]; they are recalled as stated there, each named by its title in that paper, and the canonical compilation as described there. Reduction, the writer mgu, goal identity, the transaction closure and the transactions-based multiagent transition system, which the maGLP and vmaGLP transition systems are defined over, and the construct process and the dispatcher, which the canonical compilation names, are those of [34, 38] and are not recalled; a recalled definition names each by its title in that paper.

Definition E.1 (Transition System, Computation, Run, Safe, Live, Correct [15, 28, 38]). A *transition system* is a tuple $TS = (C, c_0, T, \sim)$ where C is an arbitrary set of *configurations*, $c_0 \in C$ a designated *initial configuration*, $T \subseteq C \times C$ a set of *transitions*, each a pair $c \to c'$ of non-identical configurations $c \neq c' \in C$, and $\sim$ a partial equivalence relation on T — an equivalence on a subset $dom(\sim) \subseteq T$; we write $[t]$ for the class of $t \in dom(\sim)$ under $\sim$.

A *computation* is a (nonempty, finite or infinite) sequence of configurations $c_1, c_2, \ldots$; it is a *run* if $c_1 = c_0$, and *safe* if $c_i \to c_{i+1} \in T$ for every two consecutive configurations. We write $c \xrightarrow{*} c'$ for the existence of a safe computation from c to c' (empty if $c = c'$). A class $[t]$ is *enabled* in c if $c \to c' \in [t]$ for some c'. A run is *live* if no class $[t]$, $t \in dom(\sim)$, is enabled in every configuration of some suffix in which no member of $[t]$ occurs, and *correct* if it is safe and live.

Definition E.2 (Implementation (Refinement) [15, 28, 36, 38]). Given two transition systems $TS = (C, c_0, T, \sim)$, the *specification*, and $TS' = (C', c'_0, T', \sim')$, an *implementation of TS by TS'* is a function $\sigma : C' \to C$ with $\sigma(c'_0) = c_0$, in which case the pair (TS', σ) is *an implementation of TS*. Given a computation $r' = c'_1, c'_2, \ldots$ of TS', $\sigma(r')$ is the computation $\sigma(c'_1), \sigma(c'_2), \ldots$ with consecutive repetitions removed; a transition $c' \to d'$ of TS' with $\sigma(c') = \sigma(d')$ is a *stutter*, contributing no step to $\sigma(r')$.

Definition E.3 (Variables, Terms, Goals, Clauses [34, 38]). Let $\mathcal{V}$ denote the set of LP variables (identifiers beginning with uppercase), henceforth called *writers*, and $\mathcal{V}? = \{X? \mid X \in \mathcal{V}\}$ the *readers*;

$\hat{\mathcal{V}} = \mathcal{V} \cup \mathcal{V}?$, and a writer X and its reader $X?$ form a *variable pair*. Terms, unit goals, goals, and clauses are as in LP, over $\hat{\mathcal{V}}$. A term, goal, or clause satisfies the *single-occurrence (SO) invariant* if every variable occurs in it at most once, and a clause satisfies the *single-reader/single-writer (SRSW) restriction* if it satisfies SO and a variable occurs in it iff its paired variable does.

Definition E.4 (Guarded Clause, Procedure, GLP Program [34, 38]). A *guarded clause* has the form H `:-` G `|` B, where H is the head, G a conjunction of guard predicates, and B the body; guard arguments are readers paired to head writers. A *GLP program* is a finite sequence of guarded clauses, each satisfying SRSW; the clauses for the same predicate form a *procedure*. The set of goals $\hat{\mathcal{G}}(M)$ of a program M includes all goals over $\hat{\mathcal{V}}$ and the vocabulary of M that satisfy SO.

Definition E.5 (Asynchronous Resolvent [34, 38]). Given a GLP program M, an *asynchronous resolvent* over M is a pair (G, σ) where $G \in \hat{\mathcal{G}}(M)$ and σ is a readers substitution.

Definition E.6 (Variables of an Agent [34, 38]). Let $\mathcal{V}_p \subset \mathcal{V}$, for $p \in \Pi$, be pairwise disjoint infinite sets of writers. The *variables of agent* p are $\hat{\mathcal{V}}_p = \mathcal{V}_p \cup \{X? \mid X \in \mathcal{V}_p\}$.

Definition E.7 (maGLP Transactions [34, 38]). Given a GLP program M and agents $P \subset \Pi$, the *maGLP transactions* over asynchronous resolvents over M are the following.

Reduce p**.** $A \in G_p$ is a unit goal, and $C \in M$ is the first clause for which the reduction of A with C (Definition "Reduction" of [38]) succeeds, with result $(B, \hat{\sigma})$, C renamed into $\hat{\mathcal{V}}_p$.

$$\frac{A \in G_p \qquad C \text{ the first clause reducing } A, \text{ with result } (B, \hat{\sigma})}{(G_p,\ \sigma_p) \ \rightarrow\ ((G_p \setminus \{A\} \cup B)\hat{\sigma},\ \sigma_p \circ \hat{\sigma}?)}$$

Communicate p **to** q**.** Participants $p, q \in P$, possibly $p = q$: $\{X? := T\} \in \sigma_p$ and $X?$ occurs in G_q.

$$\frac{\{X? := T\} \in \sigma_p \qquad X? \text{ occurs in } G_q}{(G_p,\ \sigma_p),\ (G_q,\ \sigma_q) \ \rightarrow\ (G_p,\ \sigma_p \setminus \{X? := T\}),\ (G_q\{X? := T\},\ \sigma_q)}$$

Where $p = q$ the transaction is unary: $(G_p,\ \sigma_p) \rightarrow (G_p\{X? := T\},\ \sigma_p \setminus \{X? := T\})$. Communicate removes the assignment it communicates [34].

Cold-call p **to** q**.** Participants $p \neq q \in P$: the network output stream in (G_p, σ_p) has a new message $\mathtt{msg}(q, X)$; (G'_p, σ'_p) is the result of advancing the network output stream and (G'_q, σ'_q) the result of adding $\mathtt{msg}(q, X)$ to the network input stream.

$$\frac{\text{the network output stream of } p \text{ has a new } \mathtt{msg}(q, X)}{(G_p,\ \sigma_p),\ (G_q,\ \sigma_q) \ \rightarrow\ (G'_p,\ \sigma'_p),\ (G'_q,\ \sigma'_q)}$$

Definition E.8 (maGLP Transition System [34, 38]). Given a GLP program M and agents $P \subset \Pi$, the *maGLP transition system* over P and M is the transactions-based multiagent transition system (Definition "Transactions-Based Multiagent Transition System" of [38]) over P, local states being asynchronous resolvents, initial local state $(\{a_0\}, \emptyset)$ with a_0 the goal `agent(ch(_?,_),ch(_?,_))`, its anonymous variables at p of $\hat{\mathcal{V}}_p$, the transactions of Definition E.7, and the equivalence $\sim$ relating $t_1 \sim t_2$ iff both are Reduce transactions at the same agent p reducing the same goal (by identity, Definition "Goal Identity" of [38]) with the same clause, both are Communicate transactions from the same p to the same q applying the counterpart of the same writer assignment to the same goal, or both are Cold-call transactions from the same p to the same q delivering the same message.

Definition E.9 (Volitional Procedure, Interactive Type [38]). A *volitional procedure* p of arity n is declared `procedure (`T`)*`$p(T_1, \ldots, T_n)$`.`, where $T_1, \ldots, T_n$ are the types of its arguments and T, in writer or reader mode as an argument type is, is its *interactive type*. An interactive type is built from constants, `String`, `Integer`, `Real`, `Key`, `Constant`, `Module`, named types so built, tuples and unions, and `Stream` or `List` as the whole type or as an argument of a tuple, never under another `Stream` or `List`, and a `Stream` only at a position the person writes; a named type in it is recursive only if it has no mode inversion [37], and it has no type parameter.

Definition E.10 (Clause of a Volitional Procedure, Interactive Term [38]). A clause of a volitional procedure p with interactive type T has the form `(`A`)*`$p(S_1, \ldots, S_n)$ `:-` G `|` B, where the *interactive term* A is a term of type T, possibly a variable but not the anonymous variable, and it is the guarded clause $p(S_1, \ldots, S_n, A)$ `:-` G `|` B (Definition E.4), of arity n+1. A clause of a volitional procedure reads an argument at an input position, by a non-variable term at that position in its head or by a guard on it, only if every later clause of the procedure reads it too, and every call of a volitional procedure, in a clause body or in the initial goal, holds a writer at each output position.

Definition E.11 (Ordinary Clause, vGLP Program [38]). A clause is *ordinary* if it is not a clause of a volitional procedure. A *vGLP program* is a finite sequence of ordinary clauses and clauses of volitional procedures, each satisfying SRSW, a clause of a volitional procedure satisfying it as a guarded clause of arity n+1; the clauses for the same predicate form a *procedure*. The set of goals $\hat{\mathcal{G}}(M)$ of a vGLP program M includes all goals over $\hat{\mathcal{V}}$ and the vocabulary of M that satisfy SO.

Definition E.12 (vmaGLP Agent State [38]). Given a vGLP program M, an *agent state* is a triple (H_p, G_p, σ_p) of a finite set H_p of typed variables *held* by the agent's person and of assignments *shown* to them and an asynchronous resolvent (G_p, σ_p) over M (Definition E.8). A shown assignment is a record of what the person was shown, and its variables are not occurrences.

Definition E.13 (Reduce and Communicate in vmaGLP [38]). Reduce applies to a unit goal that is not an n-ary goal of a volitional procedure, and a clause of a volitional procedure is taken as its guarded clause (Definition E.10). Communicate delivers an assignment $\{X? := T\} \in \sigma_p$ to a reader $X? \in H_q$ once $\hat{T}$, the term T with each reader replaced, repeatedly, by the term assigned to it in the configuration, has no reader, and then delivers $\hat{T}$: it removes $X?$ from H_q and adds the variables of $\hat{T}$, each with the type of its position in $\hat{T}$, and the assignment, shown, the assignments used in forming $\hat{T}$ being communicated with it; Reduce, Cold-call and every other Communicate leave the held variables unchanged.

Definition E.14 (Ask [38]). The *transactions on the person's variables* at an agent p are *Ask* of this definition and *Answer* of the next. Ask p: A is an n-ary goal of a volitional procedure q with interactive type T, X a writer of $\mathcal{V}_p$ occurring nowhere in the configuration,

the *interactive variable* of the asked goal, of type T' where T is T' or T'?; the goal takes its end of X and the person the other: $(A', Y) = (q(S_1, \ldots, S_n, X?), X)$ where T is in reader mode, and $(q(S_1, \ldots, S_n, X), X?)$ where it is in writer mode. The asked goal keeps the identifier of A.

$$\frac{A = q(S_1, \ldots, S_n) \in G_p \qquad X \text{ fresh} \qquad (A', Y) \text{ as above}}{(H_p,\ G_p,\ \sigma_p) \ \rightarrow\ (H_p \cup \{Y\},\ G_p \setminus \{A\} \cup \{A'\},\ \sigma_p)}$$

Definition E.15 (Answer [38]). Answer $p{:}\ X \in H_p$ is a writer whose reader $X?$ occurs in the configuration, and t a term of the type of X, ground but for variables of $\hat{\mathcal{V}}_p$ occurring nowhere in the configuration: a reader $Y_i?$ at each position the person writes of type `Stream(T')` or `List(T')`, unless the person ends the list there with `[]`, and a writer W_j at each outermost position the program writes; where X itself is of such a type, t is $[e \mid Y_1?]$ with e such a term of type T', or `[]` for a list. The person holds Y_i and $W_j?$, each with the type of its position in t.

$$\frac{X \in H_p \qquad t \text{ as above}}{(H_p,\ G_p,\ \sigma_p) \ \rightarrow\ (H_p \setminus \{X\} \cup \{Y_1, \ldots, Y_k, W_1?, \ldots, W_m?\},\ G_p,\ \sigma_p \cup \{X? := t\})}$$

Definition E.16 (vmaGLP Transition System [38]). Given a vGLP program M and agents $P \subset \Pi$, the *vmaGLP transition system* over P and M is the transactions-based multiagent transition system (Definition "Transactions-Based Multiagent Transition System" of [38]) over P, local states being agent states, initial local state $(\emptyset, \{a_0\}, \emptyset)$ with a_0 as in Definition E.8, the transactions being the Reduce, Communicate and Cold-call transactions of maGLP as Definition E.13 adapts them and the transactions on the person's variables, Ask and Answer (Definitions E.14 and E.15), read as those of Definition E.7, and the equivalence $\sim$ below. $\sim$ is the partial equivalence whose domain is the Ask, Reduce, Communicate and Cold-call transactions, relating two Reduce, Communicate or Cold-call transactions as maGLP does, the person counting as a goal, and $t_1 \sim t_2$ iff both are Ask transactions at the same agent p asking the same goal (by identity, Definition "Goal Identity" of [38]).

Definition E.17 (Proper Run [38], adapted from [34]). A vmaGLP run is *proper* if, at each agent p, a variable that occurs in G_p after a transition but not before it also does not occur in G_p or in H_p at any earlier configuration, and the clause each Reduce renames is renamed apart from every agent's goals, held variables and readers substitution, in the configuration it reduces and in every earlier one.

Definition E.18 (Question, Open, Closed, Volition [38]). In a configuration of the vmaGLP transition system, a writer $X \in H_p$ is a *question* of p's person: an interactive variable the person writes, introduced by an Ask, a writer inside the program's output delivered to the person, or the writer of a stream or list that an Answer leaves to the person, the tail of the stream or list answered or one inside the answer; it is *open* if its reader $X?$ occurs in the configuration, and *closed* otherwise. A *volition* of p's person is an Answer transition of p: the person's answer to a question.

Theorem E.19 (Volitional Soundness [38]). *Let $c^1 \rightarrow c^2 \rightarrow \cdots$ be a proper safe run of the vmaGLP transition system over P and M, $p \in P$, and X a question of p's person in c^i. Then the writer X occurs in c^i in H_p and nowhere else, and a transition $c^i \rightarrow c^{i+1}$ that adds an assignment $X? := t$ to a readers substitution is the Answer of p to X.*

The proof is in [38].

Definition E.20 (Person Channel, Person Writer, GLP with Persons, Grant [38]). Let N be a GLP program and $P \subset \Pi$. A *person channel* of a goal G_0 satisfying SO is a variable pair $(U, U?)$ such that $U?$ occurs in G_0 and U does not; U is its *person writer*. *GLP with persons* over P and N, with a designated person channel of each agent's initial resolvent, is the transition system obtained from the maGLP transition system over P and N (Definition E.8) by adding, at each agent p, *grant* transitions

$$(G_p, \sigma_p) \ \rightarrow\ (G_p,\ \sigma_p \circ \{U? := [c \mid U_1?]\})$$

where c is a ground term `input(`*Id*`,` *Pos*`,` *R*`)` or `refresh`, U the person writer of p, and U_1 a writer of $\mathcal{V}_p$ occurring nowhere in the configuration, the person writer after the transition, every other agent stationary; the partial equivalence is the maGLP transaction equivalence, with grant transitions outside its domain.

The canonical compilation [38]. Let M be a vGLP program. A procedure of M *reaches a question* if it is volitional or a clause of it calls a procedure that does; each such procedure has one more argument, last, its *ask stream*, which its goal writes and the dispatcher reads. In a clause of such a procedure, each call in the body of a procedure that reaches a question is given a fresh writer as its ask stream, and the head carries $D?$, where D is the clause's ask stream, with the body's ask streams merged into D: the call is given D itself where there is one such call, `merge` goals merge them where there are more, and the head carries `[]` in place of $D?$ where there is none.

For a volitional procedure q of M with interactive type T, the *asking clause* of q, under the name q so that every call of q reaches it unchanged, is

```
q(S1, ..., Sn, [ask(T, t(X))|D?]) :- q1(S1', ..., Sn', X?, D).
```

where S_l' passes the argument on: at an input position the head carries S_l and S_l' is its reader $S_l?$, and at an output position the head carries $S_l?$ and S_l' is the writer S_l. `X` and `X?` are exchanged where T is in writer mode. `t` is the name of T in lowercase, the names of its type arguments, so formed, appended, with `_w` for writer mode and `_r` for reader mode, as in `approval_w` and `stream_request_r`. Where two modules of M define types of one name, the name of each is prefixed with the path of its module, the names of its directories and its file joined by `_`, as in `parent_approval_w`. So the asks of M are of one type, the union over those functors.

The *canonical compilation* $\lceil M \rceil$ of a vGLP program M is the GLP program consisting of:

- every ordinary clause of M, extended with its ask streams (above);
- for each volitional procedure q of M, its clauses as guarded clauses of arity n+1 under a fresh name, `q1` above, extended, and its asking clause;
- for each interactive type T of M, the *construct process* of T, which the dispatcher spawns on an ask with its variable and its channels to the dispatcher ([38]);
- the *dispatcher*, which reads the ask stream of the initial goal and the person channel, spawns a construct process with a

fresh identifier on each ask, and routes the person's grants to the construct processes by identifier, by the standard stream techniques ([38] prints it).

The initial goal of $\lceil M \rceil$ at each agent is that of M, extended, and the dispatcher on the ask stream of that goal and on the person channel, which is its person side.

A vGLP program is organised in modules as a GLP program is [37]. A `.vglp` source in the directory tree of a program is compiled into a module of that program, as a `.glp` source is, and the `self.vglp` of a directory holds the procedures, volitional or not, and the widget declarations shared by every module in that directory and below it, as its `self.glp` holds those of GLP; the `self.vglp` of the root holds widget declarations only. A procedure that reaches a question is exported, imported and shared as any procedure is, with its ask stream added, so the questions of a program are those of all its modules. The compilation forms the type of the asks, the construct processes and the dispatcher for the linked program, and the GLP module that forms the program's initial goal imports the compiled procedures it calls, ask streams included, and the dispatcher. The compilation is typed: $\lceil M \rceil$ carries the type definitions and procedure declarations of M, each procedure that reaches a question declared with its ask stream added, a volitional procedure's declaration being its asking clause's and its guarded clauses declared under their fresh name with the argument of its interactive type added in the mode of that type, the type of the asks, and the declarations of the construct processes and the dispatcher. Every module of $\lceil M \rceil$ type-checks as GLP [37] with no type written by hand: the compilation writes into each module that imports a compiled procedure, the dispatcher, or a procedure whose declaration names a type the compilation forms, the types it forms for them, the linker so writing them, among them the type of the asks, the view types and the parameter of the person channel ([38]). A GLP module that calls a compiled procedure is a module of the vGLP program, every GLP program being a vGLP program, and is compiled with it.

A vGLP program declares the type of its output to its person by defining the type `Output`, a reserved name; the compiled program draws that output, construct `0`, by the widget of its type `Stream(Output)`, by default a thread whose elements are drawn by the widget of `Output` (Definition "Default Widget" of [38]), and the dispatcher merges these draws with the constructs'. A program that writes nothing to its person defines no `Output` and draws no construct `0`. A program that writes to its person and defines no `Output` is refused, and so is one whose `Output` has a position the person writes. Construct `0` is drawn when the program starts, and the dispatcher numbers the questions from 1.

The asking clause passes each argument of the goal on through a pair of its own, so until the pair's assignment is communicated a compiled clause that reads the argument suspends; a later clause that does not read it could then be taken where the source goal takes the earlier one, and the restriction on the clauses of a volitional procedure (Definition E.10) excludes it. At an output position the asking clause's head carries a reader, which a writer of the goal matches and a term of the goal that is not a writer does not, so the calls of a volitional procedure hold writers there (Definition E.10), and a goal keeps them until it is asked, holding their one occurrence.

Definition E.21 (Compilation Mapping: Goals [38]). Given a configuration of GLP with persons over P and $\lceil M \rceil$, the mapping σ yields the vmaGLP configuration over P and M with, at each agent, the goals G of this definition and the held variables H and the readers substitution σ_r of the next two: G is the multiset of the agent's goals, each read back to its source with its ask stream dropped: $q(S_1, \ldots, S_n, D)$ as the n-ary goal $q(S_1, \ldots, S_n)$, $q_1(S_1, \ldots, S_n, X?, D)$ and $q_1(S_1, \ldots, S_n, X, D)$ as the asked goal, with the identifier of the goal its asking clause reduced, and any other compiled goal as itself; the construct processes, the dispatcher and the `merge` goals the compilation adds contributing nothing; and in every goal an asking clause's pair for an argument read as that argument: at an input position $S_l?$ as the term u, until the assignment is communicated, and at an output position S_l as W and $S_l?$ as $W?$.

Definition E.22 (Compilation Mapping: Held Variables [38]). H is the set of variables held for the person, each with its type: by an ask the dispatcher has not yet taken, and by a construct process that has not drawn its question, the person's end of the interactive variable; by a construct process that has drawn it, the writers of the positions the person writes, and the readers of the positions the program writes inside the person's answers whose output it has not drawn; the writer of each question a construct process removed as widowed, or bound after its reader was dropped; and the output the construct processes have drawn, shown, each reader in it replaced by its value.

Definition E.23 (Compilation Mapping: Substitution [38]). σ_r is the restriction of the readers substitution to the readers of *source variables* — the variables of the initial goal, of the compiled clauses of M as renamed, but for their ask streams, the interactive variables, and the variables of the terms construct processes bind questions with — less the assignment to the reader of a question bound after its reader was dropped, and less the assignments the asking clauses' reductions make, but for the assignment $W? := S_l?$ of an output position, which it reads through, absent while $S_l?$ is unassigned and $W? := u$ once $S_l? := u$ is; and with each assignment to a reader a construct process holds that a Communicate has delivered to it and the construct process has not yet drawn, Communicate removing the assignment it delivers (Definition E.7).

Lemma E.24 (Compilation, per Transition [38]). *Let σ be the mapping of Definitions E.21, E.22 and E.23 and $c \rightarrow c'$ a transition of GLP with persons over P and $\lceil M \rceil$ at agent p. A grant, a reduction of the dispatcher or of a* `merge` *goal the compilation adds, a reduction of a construct process that neither binds a question nor draws output, a Communicate on the person channel, on an ask stream, or between the dispatcher and the construct processes, and a Communicate to a construct process are stutters. A reduction of the asking clause of q maps to the Ask of the source goal. A Communicate of an assignment an asking clause's reduction made is a stutter. A construct process binding a question X with the term t its person's input forms, a term of the type of X ground at the positions the person writes but for the stream and list positions, which hold readers of new writers it holds or, for a list the person ends,* `[]`*, with new writers at the positions the program writes, whose readers it holds, maps to the Answer of p to X with t. A construct process's draw of output it holds, every reader in it assigned, maps to the source Communicate of that output to the person. A reduction of a compiled clause of M maps to the Reduce of*

the source goal with the source clause. A Communicate to a goal of the program, within the agent or from another, maps to the source Communicate of the same assignment, and a Cold-call to the source Cold-call.

The lemma is the safety case of the proof of the Compilation theorem of [38], stated transition by transition.